

**Einfluss der Entwicklerkompetenz auf
die Effektivität von
KI-Assistenzsystemen:
Eine empirische Untersuchung von
Vibe-Coding in der
Unternehmenssoftwareentwicklung**

Cihad Gök

Dezember 2025

Inhaltsverzeichnis

1	Einleitung	6
1.1	Motivation	6
1.2	Problemstellung	7
1.3	Zielsetzung der Arbeit	7
1.4	Aufbau der Arbeit	9
2	Theoretischer Hintergrund und Stand der Forschung	10
2.1	KI-Assistenzsysteme und das Paradigma des Vibe-Coding	10
2.1.1	Technologische Grundlagen: Transformer und In-Context Learning	10
2.1.2	Definition: Vibe-Coding als System-1-Aktivität	11
2.1.3	Theoretische Einordnung in die HCI-Forschung	11
2.2	Kompetenzmodelle: Vom Dreyfus-Modell zu kognitiven Architekturen . .	12
2.2.1	Das Dreyfus-Modell der Expertise	12
2.2.2	Kritik und Erweiterung: ACT-R und Adaptive Expertise	12
2.2.3	Situated Learning und Cognitive Apprenticeship	13
2.3	Kognitionspsychologische Aspekte der Human-AI Interaction	13
2.3.1	Automation Bias und Vertrauen	13
2.3.2	Cognitive Load Theory (CLT)	13
2.3.3	Mixed-Initiative Interaction	14
2.4	Empirische Studien zu KI-Assistenz	14
2.5	Softwarequalität und Produktivität	14
2.5.1	Softwarequalität nach ISO/IEC 25010	14
2.5.2	Produktivität: Das SPACE-Framework	15
2.6	Zusammenfassung und Forschungslücke	15
3	Forschungsdesign und Hypothesenentwicklung	16
3.1	Methodischer Ansatz: Between-Subjects Design	16
3.2	Theoretische Herleitung der Hypothesen	16
3.2.1	H1: Der „Sweet Spot“ der KI-Unterstützung (Mid-Code)	16
3.2.2	H2: Das Validierungs-Paradoxon (Low-Code)	17
3.2.3	H3: Skeptizismus und Qualitätsfokus (High-Code)	17
3.3	Operationalisierung der Kompetenzstufen	18
3.4	Kontrolle von Störvariablen (Confounder)	20
3.5	Die Testumgebung als unabhängige Variable: Ecological Validity	20
4	Durchführung und Methodik der Datenerhebung	21
4.1	Ablauf der Untersuchung	21

4.2	Operationalisierung der Metriken	22
4.2.1	Funktionale Korrektheit (Effectiveness)	22
4.2.2	Effizienz (Efficiency)	22
4.2.3	Codequalität (Quality)	22
4.2.4	Interaktionsqualität (Interaction Quality)	22
4.3	Güte der Untersuchung (Threats to Validity)	23
4.3.1	Konstruktvalidität (Construct Validity)	23
4.3.2	Interne Validität (Internal Validity)	23
4.3.3	Externe Validität (External Validity)	23
4.3.4	Reliabilität (Reliability)	24
4.4	Datenanalyse-Strategie	24
5	Technische Realisierung der Testumgebung	25
5.1	Anforderungen an die Sandbox	25
5.2	Architektur und Tech-Stack: Kognitive Dimensionen	25
5.2.1	Backend: Node.js & Express (Layered Architecture)	25
5.2.2	Frontend: React & Vite	26
5.3	Technische Funktionsweise von Copilot in der Sandbox	26
5.3.1	Prompt Construction und Context Retrieval	26
5.3.2	Token Prediction und Ranking	27
5.4	Design der Aufgaben (Workload Grading)	27
5.5	Automatisierung der Evaluation	27
5.5.1	Das Analyse-Skript (<code>run.js</code>)	27
5.5.2	Grenzen der LLM-gestützten Code-Analyse	28
5.6	Relevanz für den Unternehmenskontext (BuyIn)	28
5.7	Containerisierung und Reproduzierbarkeit	28
5.7.1	Reproduzierbarkeit / Artefaktzugang	28
6	Ergebnisse	30
6.1	Datengrundlage und Toolkontext	30
6.1.1	Studiensetting und Rahmenbedingungen	30
6.1.2	Datenquellen und Artefakte	30
6.1.3	Dateninventar und Vollständigkeit	31
6.1.4	Zuordnung und Zusammenführung der Daten	31
6.1.5	Reproduzierbarkeit und Auswertungswerkzeuge	32
6.1.6	Abgrenzung zu Kapitel 7	32
6.2	Task-Pipeline-Analyse	32
6.2.1	Methodische Definitionen	32
6.2.2	Pipeline-Ergebnisse (T1–T5)	33
6.2.3	Beobachtungen und Interpretation	33
6.3	Detaillierte Lösungsstrategien pro Task (T1–T5)	35
6.4	Code-Qualität und technische Metriken	38
6.4.1	ESLint und Typisierung	38
6.4.2	Testabdeckung und Teststabilität	39

6.4.3	Architektur- und Strukturindikatoren	39
6.4.4	Zusammenhänge mit Task-Fortschritt	40
6.4.5	Kurzfazit: Beobachtete Qualitätsmerkmale	40
6.5	Chat-Interaktionsmuster	40
6.5.1	Umfang und Intensität der Interaktion	41
6.5.2	Struktur der Prompts	41
6.5.3	Debugging-Interaktionen und Iterationsmuster	42
6.5.4	Modelltypische Antwortmuster (Claude Sonnet 4.5)	42
6.5.5	Beziehung zwischen Chat-Mustern und Task-Fortschritt (deskriptiv)	43
6.6	Triangulation der Datenquellen	43
6.6.1	Datenbasis und Aggregationslogik	44
6.6.2	Pipeline-Fortschritt und Implementationsstrategien (Strukturindikatoren)	44
6.6.3	Pipeline-Fortschritt und technische Metriken	45
6.6.4	Pipeline-Fortschritt und Chat-Interaktionsmetriken	45
6.6.5	Pipeline-Fortschritt und Survey-Merkmale	46
6.6.6	Abgrenzung zu Kapitel 7	46
6.7	Zusammenfassung der Ergebnisse	46
7	Diskussion	49
7.1	Ziel und Abgrenzung der Diskussion	49
7.2	Rückbindung an die Hypothesen	50
7.2.1	H1: Mid-Code als Sweet Spot	51
7.2.2	H2: Validierungsparadoxon bei Low-Code	51
7.2.3	H3: Skepsis und Qualitätsfokus bei High-Code	52
7.3	Kompetenzabhängige Interaktionsmuster	52
7.3.1	No-Code	53
7.3.2	Low-Code	53
7.3.3	Mid-Code	53
7.3.4	High-Code	54
7.4	Interpretation der Pipeline-Muster	54
7.4.1	Attempted ist nicht gleich Implemented _{static}	54
7.4.2	Implemented _{static} ist nicht gleich Verified	54
7.4.3	Warum Verified gleich null kein eindeutiges Negativsignal ist	55
7.5	Einordnung der Chat-Interaktionsmuster	55
7.5.1	Warum Chat-Intensität kein Effizienzmaß ist	55
7.5.2	Warum Debugging-Loops normal sind	55
7.5.3	Explorative und zielgerichtete Interaktion	56
7.6	Implikationen für Praxis und Forschung	56
7.6.1	Implikationen für Unternehmen	56
7.6.2	Implikationen für die Evaluation von KI-Coding-Tools	56
7.6.3	Abgrenzung zu Marketing-Narrativen	57
7.7	Limitationen und Ausblick	57

8	Fazit und Schlussfolgerungen	58
8.1	Ziel und Einordnung des Kapitels	58
8.2	Beantwortung der Forschungsfragen	58
8.2.1	Forschungsfrage 1: Kompetenz und Pipeline-Status	58
8.2.2	Forschungsfrage 2: Code- und Test-Artefakte	59
8.2.3	Forschungsfrage 3: Chat-Interaktionsmuster	59
8.3	Zentrale Erkenntnisse der Arbeit	59
8.4	Beitrag der Arbeit	60
8.5	Abschließendes Fazit	61

1 Einleitung

1.1 Motivation

Große Sprachmodelle (Large Language Models, LLMs) werden zunehmend in der Softwareentwicklung eingesetzt und unterstützen Entwurf, Implementierung und Wartung. Im Folgenden wird ein solches Assistenzsystem als *KI-Assistenzsystem* bezeichnet (engl.: *AI Coding Agent*) und konsistent so verwendet. In vielen Arbeitsabläufen verschiebt sich der Schwerpunkt von der reinen Codeerzeugung hin zur Bewertung, Auswahl und Integration vorgeschlagener Änderungen. Damit gewinnt eine Arbeitsweise an Relevanz, die häufig als *Vibe-Coding* beschrieben wird. Entwicklerinnen und Entwickler delegieren Teilaufgaben an ein KI-Assistenzsystem und übernehmen verstärkt Aufgaben der semantischen Prüfung, des Reviews sowie der Einbettung in bestehende Strukturen. Diese Form der Mixed-Initiative-Zusammenarbeit betont weniger syntaktische Details als vielmehr das Verständnis von Anforderungen, Nebenbedingungen und Systemkontext.

Für Unternehmen ist diese Entwicklung insbesondere im Brownfield-Kontext bedeutsam. Ein Großteil praktischer Softwareentwicklung findet in bestehenden Codebasen statt, die technische Schulden, heterogene Architekturen, historische Entscheidungen und operative Randbedingungen aufweisen. Die produktive Integration von KI-Assistenzsystemen in etablierte Entwicklungsprozesse muss daher neben Effizienzgewinnen auch Qualitätssicherung, Nachvollziehbarkeit und Risikobegrenzung berücksichtigen. Daraus ergibt sich ein Spannungsfeld zwischen schneller Iteration und belastbarer Validierung.

Für die vorliegende Arbeit ist dabei zentral: Potenzielle Effizienzgewinne werden nicht quantifiziert; im Fokus steht vielmehr, wie Integration, Validierung und Review im Zusammenspiel mit einem KI-Assistenzsystem empirisch beobachtbar werden.

Wissenschaftlich leistet die Arbeit damit einen Beitrag zur empirischen Evaluation von KI-Assistenzsystemen im Brownfield-Kontext, der über reine Produktivitätsbetrachtungen hinausgeht.

Vor diesem Hintergrund ist der Nutzen eines KI-Assistenzsystems in dieser Arbeit nicht als reine Eigenschaft des Tools zu verstehen, sondern als Zusammenspiel aus (i) Interaktion zwischen Mensch und System, (ii) entwicklerseitiger Kompetenz und (iii) Praktiken der Validierung und Integration. Automation Bias kann dazu beitragen, dass Vorschläge überbewertet oder unzureichend geprüft werden, während umgekehrt übermäßige Skepsis potenzielle Vorteile reduziert. Eine wissenschaftlich belastbare Einordnung erfordert daher eine Betrachtung. Diese verknüpft Kompetenz als Analyseperspektive mit beobachtbaren Artefakten und Ergebnissignalen der Arbeit am Code, ohne daraus kausale Effekte oder Tool-Rangordnungen abzuleiten.

1.2 Problemstellung

In der Diskussion um KI-Assistenzsysteme werden häufig Produktivitätsannahmen formuliert, ohne hinreichend zu differenzieren, wie stark die Interaktion durch Kompetenz, Arbeitsstil und Validierungspraktiken moderiert wird. Unter realitätsnahen Bedingungen bestehender Codebasen und unter Zeitdruck bleibt zudem unklar, in welchem Verhältnis wahrgenommener Fortschritt, implementierte Änderungen und verifiziertes Verhalten zueinander stehen. Damit entsteht eine Forschungslücke: Kontrollierte Untersuchungen kombinieren selten Brownfield-Settings, eine differenzierte Ergebnislogik entlang einer Pipeline und Kompetenz als explizite Analyseperspektive.

Diese Arbeit ist daher keine Produktivitätsmessung im Sinne von Output-Geschwindigkeit, Codeumfang (z.B. Zeilen Code) oder Zeitersparnis, sondern eine Analyse von Integrations-, Validierungs- und Reviewarbeit im Vibe-Coding-Kontext.

Sie trifft keine Aussagen über kausale Effekte und leitet keine Rangordnung von Tools oder Modellen ab.

Eine zentrale methodische Herausforderung liegt in der Messbarkeit von Effektivität. Chat-Interaktionen und Planäußerungen im Assistenzdialog sind nicht gleichzusetzen mit funktionsfähiger Implementierung. Ebenso ist eine Implementierung im Code nicht gleichzusetzen mit testgebundener Bestätigung, dass gewünschtes Verhalten tatsächlich erreicht wurde. Diese Differenzen sind im Kontext von KI-generiertem Code besonders relevant, weil die Geschwindigkeit der Generierung und die Plausibilität der Vorschläge die Notwendigkeit strukturierter Verifikation nicht aufheben, sondern eher erhöhen können.

Zur Bearbeitung dieser Messprobleme wird eine Diagnoselogik benötigt, die Ergebnisse schrittweise differenziert und durch mehrere Evidenzquellen absichert. In dieser Arbeit wird dafür eine Pipeline-Logik verwendet, die den Bearbeitungsstand einer Aufgabe als *Attempted*, *Implemented_{static}* oder *Verified* beschreibt. Für die Trichterdarstellung wird in Kapitel 6 zusätzlich eine abgeleitete Evidenzstufe *Implemented** verwendet. Diese Status sind als Diagnose- und Ergebnislogik zu verstehen, nicht als Qualitätsurteil. Die Einordnung wird nicht aus einer einzelnen Quelle abgeleitet, sondern trianguliert über Code-Artefakte, Test-Artefakte, Chat-Logs und Survey-Daten. Ziel ist eine methodisch robuste, deskriptive Einordnung, die subjektive oder dialogbasierte Signale nicht mit verifiziertem Outcome gleichsetzt und den Interaktionskontext dennoch berücksichtigt.

1.3 Zielsetzung der Arbeit

Ziel dieser Arbeit ist die Evaluation eines KI-Assistenzsystems im Vibe-Coding-Kontext in einer kontrollierten Brownfield-Sandbox. Zur begrifflichen und methodischen Trennung wird zwischen der Assistenzumgebung GitHub Copilot (Tool), dem zugrundeliegenden LLM Claude Sonnet 4.5 und dem KI-Assistenzsystem als funktionaler Einheit unterschieden. Beobachtungen beziehen sich je nach Datenquelle auf unterschiedliche Ebenen. Es werden keine Tool- oder Modellvergleiche durchgeführt und keine Rangordnung abgeleitet. Die Evaluation erfolgt in einer kontrollierten Coding-Challenge mit $N = 18$ Teilnehmenden, einer Timebox von 60 Minuten und einer standardisierten, containerisierten

Arbeitsumgebung. Die Aufgaben umfassen acht Tasks (T1–T8). In der Ergebnisanalyse werden insbesondere frühe Tasks (T1–T5) berücksichtigt, da das Zeitlimit die Bearbeitungsreihenfolge und -tiefe strukturell beeinflusst.

Gegenstand der Arbeit ist dabei ein *interaktives* KI-Assistenzsystem mit kontinuierlicher menschlicher Kontrolle (Human-in-the-Loop); nicht betrachtet werden autonome Agenten mit selbstständiger Aufgabenplanung, Tool-Auswahl oder Entscheidungsautonomie.

Die Arbeit leistet im Kern vier Beiträge:

- (a) Entwicklung und Verwendung einer Brownfield-Sandbox als Messumgebung für kontrollierte, aber realitätsnahe Aufgaben in einer bestehenden Codebasis.
- (b) Operationalisierung von Aufgabenerfolg über die Pipeline-Status *Attempted*, *Implemented_{static}* und *Verified* (sowie *Implemented** als abgeleitete Evidenzstufe) als Trichter- und Diagnoselogik.
- (c) Analyse der Ergebnisse im Kontext von Kompetenzgruppen als Analyseperspektiven (No-Code, Low-Code, Mid-Code, High-Code), ohne diese Einteilung als Wertung von Personen zu interpretieren.
- (d) Methodische Triangulation über Code-Artefakte, Test-Artefakte, Chat-Logs und Survey, um die Differenz zwischen intendiertem Vorgehen, implementierten Änderungen und verifiziertem Verhalten empirisch einordnen zu können.

Forschungsfragen. Aus diesen Zielsetzungen ergeben sich drei Forschungsfragen:

- **RQ1:** Wie hängt die Entwicklerkompetenz als Analyseperspektive mit dem Erreichen der Pipeline-Status *Attempted*, *Implemented_{static}* und *Verified* (sowie der abgeleiteten Evidenzstufe *Implemented**) in einer kontrollierten Brownfield-Sandbox zusammen?
- **RQ2:** Wie tragen Code- und Testartefakte zur Ergebnisbewertung entlang der Pipeline-Status bei, insbesondere zur Trennung von Implementierung und testgebundener Verifikation?
- **RQ3:** Welche Chat-Interaktionsmuster treten im Vibe-Coding-Kontext auf, und wie sind sie für die Evaluation eines KI-Assistenzsystems im Hinblick auf Validierung und Integration einzuordnen?

Die Kompetenzgruppen dienen dabei ausschließlich der analytischen Typisierung; daraus wird keine Rangordnung abgeleitet und die Einleitung nimmt keine Ergebnisse vorweg. Die Aussagen dieser Arbeit beziehen sich auf Branches als Analyseeinheiten und auf aggregierte Muster über Gruppen hinweg, nicht auf eine Leistungsbewertung einzelner Personen. Ergänzend werden theoriegeleitet Hypothesen entwickelt, um erwartbare Muster im Spannungsfeld zwischen Unterstützungspotenzial, Validierungsaufwand und Interaktionsdynamik strukturiert zu prüfen, ohne Ergebnisse vorwegzunehmen.

1.4 Aufbau der Arbeit

Kapitel 2 führt die theoretischen Grundlagen ein und ordnet Vibe-Coding als Arbeitsstil in relevante Perspektiven ein, darunter Kompetenzmodelle, Human-Computer Interaction, Cognitive Load, Automation Bias sowie etablierte Bezugsrahmen aus Qualitäts- und Produktivitätsdiskursen (u. a. ISO/SPACE). Kapitel 3 beschreibt das Forschungsdesign, die Herleitung der Hypothesen und die Operationalisierung zentraler Konstrukte, einschließlich der Kompetenzgruppen als Analyseperspektiven und der Pipeline-Status *Attempted*, *Implemented_{static}* und *Verified* (sowie *Implemented**). Kapitel 4 erläutert die Durchführung der Studie, die Datenerhebung und die verwendeten Metriken sowie die Absicherung der Validität durch die Kombination mehrerer Datenquellen.

Kapitel 5 dokumentiert die technische Brownfield-Sandbox, ihre Architektur und die Automatisierungs- und Auswertungsinfrastruktur, die eine standardisierte, reproduzierbare Messung im kontrollierten Setting ermöglicht. Kapitel 6 präsentiert die Ergebnisse in deskriptiver Form und strukturiert sie entlang der Pipeline-Logik und der triangulierten Evidenz aus Artefakten und Interaktionen. Kapitel 7 diskutiert die Befunde, ordnet sie in den theoretischen Kontext ein und leitet Implikationen sowie Limitationen ab, ohne kausale Schlussfolgerungen über die beobachteten Zusammenhänge hinaus zu beanspruchen. Kapitel 8 schließt die Arbeit mit einer synthetischen Beantwortung der Forschungsfragen und einer Einordnung der Beiträge ab.

Abgrenzung und wissenschaftlicher Beitrag. Im Unterschied zu Teilen der bestehenden Forschung steht in dieser Arbeit weder die Optimierung von Prompts noch der Vergleich von Tools oder Modellen im Vordergrund. Ebenso werden keine Produktivitätsmetriken im Sinne von Ausstoß, Geschwindigkeit oder Zeitersparnis als primäre Zielgrößen bewertet. Stattdessen wird Vibe-Coding im Brownfield-Setting als Interaktions- und Integrationsprozess untersucht, der sich nicht zuverlässig über einzelne Proxy-Metriken abbilden lässt. Der Beitrag liegt in einer evaluativen Perspektive auf Nutzungstypen (Kompetenzgruppen als Analyseperspektiven), eine evidenzbasierte Ergebnislogik über Pipeline-Status sowie die systematische Triangulation über Code-, Test-, Chat- und Survey-Artefakte. Damit wird sichtbar gemacht, wie sich intendiertes Vorgehen, implementierte Änderungen und testgebunden verifiziertes Verhalten unter Zeitdruck zueinander verhalten, ohne daraus eine Leistungsbewertung einzelner Personen oder eine Rangordnung von Assistenzsystemen abzuleiten.

2 Theoretischer Hintergrund und Stand der Forschung

Dieses Kapitel legt das theoretische Fundament für die Untersuchung der Interaktion zwischen menschlicher Kompetenz und KI-Assistenzsystemen. Es erläutert zunächst das Konzept der *KI-Assistenzsysteme* und grenzt den neuartigen Arbeitsmodus des *Vibe-Coding* von klassischer Softwareentwicklung ab (Abschnitt 2.1). Darauf aufbauend wird ein Kompetenzmodell eingeführt, das auf dem Dreyfus-Modell der Expertise basiert, jedoch kritisch um moderne kognitive Architekturen wie ACT-R erweitert wird (Abschnitt 2.2). Anschließend werden kognitionspsychologische Aspekte der Mensch-Maschine-Interaktion – insbesondere *Dual-Process Theory*, *Cognitive Load* und *Automation Bias* – beleuchtet, die erklären, warum unterschiedliche Kompetenzstufen unterschiedlich stark von KI profitieren (Abschnitt 2.3). Abschließend werden Qualitäts- und Produktivitätsmodelle (ISO 25010, SPACE) vorgestellt, die als Messinstrumente dienen (Abschnitt 2.5).

2.1 KI-Assistenzsysteme und das Paradigma des Vibe-Coding

Die Integration von *Large Language Models (LLMs)* in die Softwareentwicklung markiert einen Paradigmenwechsel. Während klassische IDE-Features wie IntelliSense deterministisch und syntaktisch operieren, arbeiten KI-Assistenzsysteme probabilistisch und semantisch.

2.1.1 Technologische Grundlagen: Transformer und In-Context Learning

Moderne Assistenzumgebungen wie **GitHub Copilot** basieren auf der Transformer-Architektur (Vaswani u. a. 2017), die es ermöglicht, Abhängigkeiten in Sequenzen über lange Distanzen zu modellieren. Technisch relevant für die Interaktion ist das *Fill-In-The-Middle (FIM)* Paradigma (Bavarian u. a. 2022). Das Modell sagt nicht nur das nächste Token voraus, sondern berücksichtigt den Kontext *vor* und *nach* der Cursor-Position.

Zusätzlich nutzen diese Systeme *Retrieval-Augmented Generation (RAG)*-ähnliche Mechanismen. Die Assistenzumgebung scannt den lokalen Kontext (geöffnete Tabs, importierte Module) und injiziert relevante Code-Snippets in den Prompt, ohne dass der Nutzer dies explizit steuert. Dies reduziert die Notwendigkeit für manuelles Prompt

Engineering, erhöht aber die Abhängigkeit von einer sauberen Projektstruktur. Das KI-Assistenzsystem ist nur so gut wie der Kontext, den es „sieht“ (Witte u. a. 2023).

2.1.2 Definition: Vibe-Coding als System-1-Aktivität

Der Begriff *Vibe-Coding* beschreibt einen emergenten Entwicklungsstil, bei dem der Fokus von der imperativen Syntax-Erstellung zur deklarativen Intentions-Beschreibung verschiebt. Ich definiere Vibe-Coding für diese Arbeit als:

„Einen iterativen Prozess der Softwareerstellung, bei dem der Mensch primär als Architekt und Reviewer agiert, während die KI die Implementierungsdetails auf Basis von natürlichsprachlichen oder kontextuellen ‚Vibes‘ (Intentionen) generiert.“

Dieser Modus lässt sich durch die *Dual-Process Theory* von Kahneman (Kahneman 2011) erklären:

- **System 1 (Schnelles Denken):** Vibe-Coding appelliert an die Intuition. Der Entwickler tippt einen Funktionsnamen, das KI-Assistenzsystem schlägt den Body vor. Die Akzeptanz erfolgt oft heuristisch („Sieht gut aus“).
- **System 2 (Langsames Denken):** Die Validierung des Codes erfordert analytische Anstrengung. Hier liegt die Gefahr: Wenn das KI-Assistenzsystem (System 1) zu gut wirkt, wird System 2 nicht aktiviert, was zu Fehlern führt.

Dieser Shift verändert die Anforderungen fundamental:

- **Shift from Writing to Reading:** Der Anteil des Code-Lesens und -Verstehens steigt massiv an.
- **Shift from Syntax to Semantics:** Entwickler:innen müssen weniger wissen, *wie* eine Schleife syntaktisch korrekt ist, aber präziser verstehen, *was* sie logisch bewirken soll.

2.1.3 Theoretische Einordnung in die HCI-Forschung

In der Human-Computer Interaction (HCI) lässt sich Vibe-Coding als eine Form der *End-User Development* (EUD) verstehen, bei der die Barriere zur Code-Erstellung gesenkt wird. Es weist Parallelen zum *Natural Language Programming* auf, unterscheidet sich jedoch durch den *Mixed-Initiative*-Charakter (Horvitz 1999): Nicht nur der Mensch gibt Befehle, sondern die KI schlägt proaktiv Lösungen vor (*Ghost Text*). Dies erfordert eine ständige Synchronisation der mentalen Modelle zwischen Mensch und Maschine.

2.2 Kompetenzmodelle: Vom Dreyfus-Modell zu kognitiven Architekturen

Um den Einfluss der menschlichen Kompetenz zu messen, ist eine fundierte Klassifizierung notwendig.

2.2.1 Das Dreyfus-Modell der Expertise

Das *Dreyfus-Modell der Expertise* (H. L. Dreyfus und S. E. Dreyfus 1986) beschreibt den Erwerb von Fähigkeiten in fünf Stufen, von der strikten Regelbefolgung bis zur intuitiven Meisterschaft. Für diese Arbeit werden vier Stufen adaptiert:

1. **Novice (No-Code):** Befolgt strikte Regeln, hat kein kontextuelles Verständnis. Verlässt sich vollständig auf die KI als „Black Box“.
2. **Advanced Beginner (Low-Code):** Kann einfache Aufgaben lösen, erkennt aber keine übergeordneten Muster. Kann KI-Vorschläge syntaktisch, aber oft nicht semantisch bewerten.
3. **Competent (Mid-Code):** Arbeitet zielorientiert und kann Probleme in Teilprobleme zerlegen. Nutzt KI als Werkzeug zur Beschleunigung, besitzt aber genug Modellverständnis zur Validierung.
4. **Proficient/Expert (High-Code):** Handelt intuitiv und erkennt Abweichungen sofort. Nutzt KI zur Automatisierung von Boilerplate, ist aber skeptisch gegenüber komplexer Logik.

2.2.2 Kritik und Erweiterung: ACT-R und Adaptive Expertise

Das Dreyfus-Modell wird oft als zu linear kritisiert. Es vernachlässigt die *adaptive Expertise* (Hatano & Inagaki), also die Fähigkeit, Wissen auf neue Domänen zu übertragen. Hier bietet die kognitive Architektur **ACT-R** (Adaptive Control of Thought-Rational) von Anderson (1996) eine tiefere Erklärung:

- **Deklaratives Wissen:** Faktenwissen („Was ist eine Schleife?“). Novizen müssen dieses Wissen mühsam abrufen.
- **Prozedurales Wissen:** Handlungswissen („Wie schreibe ich eine Schleife?“). Experten haben dies in „Produktionsregeln“ kompiliert, die schnell und mühelos ablaufen.

Relevanz für Copilot: Das KI-Assistenzsystem liefert *prozedurale Chunks* (fertigen Code). Für Novizen, die nur über deklaratives Wissen verfügen, wirkt dies wie eine „Prothese“, die fehlende Produktionsregeln ersetzt. Für Experten ist es eine Beschleunigung bereits vorhandener Regeln.

2.2.3 Situated Learning und Cognitive Apprenticeship

Ergänzend bietet der Ansatz des *Situated Learning* (Lave und Wenger 1991) und *Cognitive Apprenticeship* (Collins u. a. 1989) eine Perspektive auf das Lernen mit KI.

- **Scaffolding:** Das KI-Assistenzsystem dient als „Gerüst“ (Scaffold), das es Lernenden ermöglicht, Aufgaben zu lösen, die knapp über ihrem aktuellen Kompetenzniveau liegen (Zone der nächsten Entwicklung).
- **Legitimate Peripheral Participation:** Novizen können durch KI-Unterstützung früher an produktiven Prozessen teilnehmen, auch wenn sie die volle Komplexität noch nicht durchdringen.

2.3 Kognitionspsychologische Aspekte der Human-AI Interaction

Der Erfolg von KI-Assistenzsystemen hängt maßgeblich von der menschlichen Komponente ab. Theorien der Mensch-Maschine-Interaktion erklären, warum Nutzende unterschiedlich auf KI-Vorschläge reagieren.

2.3.1 Automation Bias und Vertrauen

Ein zentrales Phänomen ist der *Automation Bias* (Parasuraman und Manzey 2010): Die Tendenz, automatisierten Systemen mehr zu vertrauen als der eigenen Urteilkraft. Dies manifestiert sich in zwei Fehlerarten:

- **Errors of Commission:** Der Nutzer akzeptiert einen falschen Vorschlag der KI (häufig bei Low-Code).
- **Errors of Omission:** Der Nutzer übersieht einen Fehler, weil das System keine Warnung ausgibt.

Das Vertrauen (*Trust*) muss kalibriert sein (Lee und See 2004). *Overtrust* führt zu unkritischer Übernahme, *Distrust* (oft bei Experten) zu ineffizienter manueller Überprüfung.

2.3.2 Cognitive Load Theory (CLT)

Die *Cognitive Load Theory* (Sweller 1988) unterscheidet drei Arten der kognitiven Belastung:

- **Intrinsic Load:** Die inhärente Komplexität der Aufgabe (z.B. Rekursion verstehen).
- **Extraneous Load:** Belastung durch die Darstellung oder das Werkzeug (z.B. komplexe IDE-Menüs).

- **Germane Load:** Die kognitive Kapazität, die für das Lernen und Schema-Bildung genutzt wird.

KI-Assistenzsysteme haben das Potenzial, den *Extraneous Load* (Syntax-Details) massiv zu senken. Dies setzt jedoch *Germane Load* frei, die für die Validierung genutzt werden muss. Wenn Novizen keine mentalen Modelle (Schemata) haben, können sie diese freigewordene Kapazität nicht nutzen, und der *Intrinsic Load* der Validierung überfordert sie. **Kritik:** Die CLT ist schwer in Echtzeit zu messen. Dennoch bietet sie ein robustes Erklärungsmodell für die Überforderung von Novizen trotz KI-Hilfe.

2.3.3 Mixed-Initiative Interaction

Die Interaktion mit Copilot ist ein Beispiel für *Mixed-Initiative Interaction* (Horvitz 1999). Beide Parteien (Mensch und KI-Assistenzsystem) können die Initiative ergreifen. Das KI-Assistenzsystem schlägt proaktiv Code vor (Ghost Text), der Mensch kann diesen annehmen, ablehnen oder durch Weitertippen modifizieren. Die Effizienz dieser Kollaboration hängt davon ab, wie gut der Mensch die Intention des KI-Assistenzsystems vorhersagen und steuern kann (*Theory of Mind* für die KI).

2.4 Empirische Studien zu KI-Assistenz

Aktuelle Studien bestätigen die theoretischen Annahmen teilweise, zeigen aber auch neue Phänomene:

- **Grounded Copilot:** Barke u. a. (2023) identifizierten zwei Modi der Interaktion: *Acceleration* (schnelleres Tippen bekannter Logik) und *Exploration* (Nutzen der KI, um neue APIs zu entdecken).
- **Reading Between the Lines:** Mozannar u. a. (2022) zeigten, dass das Modifizieren von KI-Code oft kognitiv teurer ist als das Selbstschreiben, wenn der Vorschlag subtile Fehler enthält.

2.5 Softwarequalität und Produktivität

Die Bewertung der Arbeit mit KI-Assistenzsystemen erfordert klare Kriterien für Qualität und Produktivität.

2.5.1 Softwarequalität nach ISO/IEC 25010

Der Standard ISO/IEC 25010 definiert Softwarequalität über acht Hauptmerkmale (*ISO/IEC 25010:2011 Systems and Software Engineering—System and Software Quality Models* 2011). Für die Bewertung von KI-generiertem Code sind insbesondere relevant:

- **Funktionale Eignung:** Erfüllt der generierte Code die fachlichen Anforderungen korrekt?

- **Wartbarkeit:** Ist der Code modular, verständlich und konsistent strukturiert? Studien zeigen, dass KI-Tools dazu verleiten können, Code zu duplizieren statt zu abstrahieren, was die Wartbarkeit langfristig gefährdet (Nadi u. a. 2022).
- **Sicherheit:** Enthält der Code Schwachstellen? Da LLMs auf öffentlichen Daten trainiert sind, können sie unsichere Muster reproduzieren (Pearce u. a. 2022).

2.5.2 Produktivität: Das SPACE-Framework

Produktivität in der Softwareentwicklung ist mehrdimensional. Das SPACE-Framework (Forsgren u. a. 2021) unterscheidet fünf Dimensionen, die auch für den Einsatz von Copilot zentral sind:

- **S (Satisfaction):** Steigert das Tool die Zufriedenheit und reduziert es Frustration?
- **P (Performance):** Führt der Einsatz zu qualitativ besseren Ergebnissen?
- **A (Activity):** Wie viele Aufgaben können in einer bestimmten Zeit abgeschlossen werden?
- **C (Communication):** Wie verändert sich die Zusammenarbeit (hier: Mensch-Maschine-Interaktion)?
- **E (Efficiency & Flow):** Wird der Arbeitsfluss durch unterbrechungsfreie Vorschläge gefördert?

2.6 Zusammenfassung und Forschungslücke

Bisherige Studien konzentrierten sich oft auf die technische Leistungsfähigkeit von Modellen oder allgemeine Produktivitätsgewinne (Peng u. a. 2023). Es fehlt jedoch an einer differenzierten Betrachtung, wie die *individuelle Kompetenz* (operationalisiert durch das Dreyfus-Modell und ACT-R) die Interaktion mit dem KI-Assistenzsystem moderiert. Insbesondere die Frage, ob KI als „Equalizer“ wirkt (Novizen auf Experten-Niveau hebt) oder die Kluft vergrößert („Rich get richer“), ist ungeklärt. Diese Arbeit adressiert diese Lücke durch ein experimentelles Design, das kognitive Theorien (Dual-Process, CLT) mit empirischen Software-Engineering-Metriken verbindet.

3 Forschungsdesign und Hypothesenentwicklung

Dieses Kapitel leitet das empirische Design der Studie aus den in Kapitel 2 vorgestellten Theorien ab. Ziel ist es, den Einfluss der unabhängigen Variable *Kompetenz* auf die abhängigen Variablen *Qualität*, *Effizienz* und *Interaktion* in einer kontrollierten Brownfield-Umgebung zu isolieren.

3.1 Methodischer Ansatz: Between-Subjects Design

Die Untersuchung folgt einem *Between-Subjects Design*, bei dem Teilnehmende basierend auf ihrer Vorkenntnis in disjunkte Gruppen eingeteilt werden. **Begründung nach Wohlin u. a. (2012):**

- **Vermeidung von Lerneffekten (Carry-Over Effects):** Ein Within-Subjects Design (jede Person löst Aufgaben einmal mit, einmal ohne KI) wäre hier problematisch, da das Lösen einer Aufgabe Wissen für den zweiten Durchgang generiert.
- **Isolierung der Kompetenzvariable:** Das Ziel ist der Vergleich der Interaktionsmuster zwischen verschiedenen Kompetenzniveaus („Mensch A + Maschine“ vs. „Mensch B + Maschine“), nicht der Vergleich des Tools an sich.

3.2 Theoretische Herleitung der Hypothesen

Die Hypothesen werden direkt aus dem Dreyfus-Modell, der Dual-Process Theory und der Cognitive Load Theory (siehe Abschnitt 2.2 und 2.3) abgeleitet.

Zur Orientierung wird die primäre Zuordnung der Hypothesen zu den Forschungsfragen explizit gemacht: H1 adressiert primär RQ1, H2 adressiert primär RQ1 und RQ2, und H3 adressiert primär RQ3. Diese Zuordnung dient ausschließlich der Strukturierung und nimmt keine Ergebnisse vorweg.

3.2.1 H1: Der „Sweet Spot“ der KI-Unterstützung (Mid-Code)

Theoretische Begründung: Nach der Cognitive Load Theory profitieren Lernende am meisten, wenn *Extraneous Load* (Syntax) reduziert wird, solange genug Schema-Wissen vorhanden ist, um den Output zu validieren. Mid-Code-Teilnehmende befinden sich in einer Phase, in der sie Konzepte verstehen, aber die syntaktische Umsetzung noch

kognitive Ressourcen bindet. Copilot übernimmt diese Last. Dadurch wird *Germane Load* für architektonische Entscheidungen frei. **Alternative Hypothese:** Die „Rising Tide“-Hypothese würde besagen, dass alle Gruppen gleichermaßen profitieren. Dies widerspricht jedoch der Theorie der *Zone der nächsten Entwicklung*, da Novizen ohne Scaffolding überfordert sind.

Hypothese 1. *Teilnehmende der Gruppe **Mid-Code** erreichen im Vergleich zu **No-Code**, **Low-Code** und **High-Code** am häufigsten den höchsten beobachteten Pipeline-Fortschritt innerhalb des 60-Minuten-Limits (operationalisiert als maximale Anzahl **implemented Tasks in evaluation/task_pipeline_v3.json**).*

3.2.2 H2: Das Validierungs-Paradoxon (Low-Code)

Theoretische Begründung: Novizen und Advanced Beginners (Dreyfus Stufe 1–2) fehlt das mentale Modell, um subtile Fehler zu erkennen. Sie operieren primär im *System 1* (intuitiv) und akzeptieren plausibel klingende Vorschläge unkritisch (*Errors of Commission*). Da ihnen die Schemata fehlen, um den Code zu „parsen“, ist der *Intrinsic Load* der Validierung zu hoch. **Empirische Stützung:** Studien von Vaithilingam u. a. (2022) zeigen, dass Nutzer oft Code akzeptieren, den sie nicht verstehen, was zu schwer findbaren Bugs führt.

Hypothese 2. *Teilnehmende der Gruppen **No-Code** und **Low-Code** erreichen im Vergleich zu **Mid-Code** und **High-Code** seltener einen hohen Pipeline-Fortschritt (operationalisiert über die Anzahl **implemented Tasks in evaluation/task_pipeline_v3.json**). Aussagen zu Sicherheitslücken, Wartbarkeit oder Overtrust werden in dieser Studie als explorativ behandelt.*

3.2.3 H3: Skeptizismus und Qualitätsfokus (High-Code)

Theoretische Begründung: Experten (Dreyfus Stufe 4–5) haben starke mentale Modelle und vertrauen der Automation weniger. Sie aktivieren *System 2* (analytisch) häufiger, um Vorschläge zu prüfen. Dies führt zu einem *Distrust*-Verhalten, bei dem Zeitersparnisse durch manuelle Reviews teilweise negiert werden. **Alternative Sichtweise:** Man könnte annehmen, Experten seien am schnellsten. Die Theorie der *Expertise Reversal Effect* (Kalyuga et al.) legt jedoch nahe, dass Hilfestellungen für Experten redundant und störend sein können.

Hypothese 3. ***High-Code** erreichen im Vergleich zu **Mid-Code** seltener den maximalen Pipeline-Fortschritt (operationalisiert über die Anzahl **implemented Tasks in evaluation/task_pipeline_v3.json**). Aussagen zu selektiver Nutzung (z.B. „Accept All“), Codier-Geschwindigkeit und Codequalität (Wartbarkeit/Sicherheit) werden in dieser Studie als explorativ behandelt.*

3.3 Operationalisierung der Kompetenzstufen

Die Kompetenzgruppen (No-Code/Low-Code/Mid-Code/High-Code) werden in dieser Arbeit ausschließlich als *Analyseperspektiven* verwendet (Stratifizierung/Segmentierung) und nicht als Leistungsbewertung einzelner Personen. Die Gruppenzuordnung erfolgt auf Basis eines transparenten, auditierbaren Scoring-Modells mit insgesamt 0–13 Punkten. Die Items werden im Pre-Survey erhoben und in Punkte gemappt; die Summe definiert die Kompetenzstufe.

Input-Quelle	Skala (Pre-Survey)	Punkte (Mapping)	Gewichtung	Begründung
Programmiererfahrung (allgemein)	Berufsjahre SWE	0J=0; 0–1=1; 1–3=2; > 3=3	3/13	Proxy für Routine; gut dokumentierbar und weniger anfällig für Ad-hoc-Interpretation.
Technologie-Fit (TS/Node/Backend)	Jahre Erfahrung mit TS/Node/Backend	0=0; <1=1; 1–2=2; > 2=3	3/13	Aufgaben sind Backend-/API-lastig; domänenspezifische Routine reduziert Setup- und Framework-Kosten.
Ausbildung / formale Grundlagen	Höchster relevanter Abschluss (Selbstauskunft)	kein/irrelevant=0; teilweise=1; einschlägig=2	2/13	Formale Grundlagen sind ein (schwach gewichteter) Indikator für systematisches Verständnis; nicht deterministisch, daher moderat gewichtet.
Selbsteinschätzung Implementations-sicherheit	Likert 1–5 mit Ankern	1–2=0; 3=1; 4=2; 5=3	3/13	Erfasst subjektive Handlungssicherheit; Anker reduzieren Interpretationsspielräume zwischen Personen.
Selbsteinschätzung Debugging/Testpraxis	Likert 1–5 mit Ankern	1–2=0; 3=1; 4=2	2/13	Debugging und Tests sind zentral für die Aufgaben (Fehlerdiagnose, Stabilisierung); bewusst getrennt von allgemeiner Skill-Selbsteinschätzung.

Tabelle 3.1: Auditierbare Scoring-Rubrik zur Kompetenz-Operationalisierung (0–13 Punkte). Die Gewichtung ist hier als maximal erreichbare Punktzahl pro Item angegeben; die Gesamtpunktzahl ist die Summe S über alle Items.

Schwellenwerte zur Gruppenzuordnung. Aus dem Gesamtscore $S \in [0, 13]$ ergeben sich die Kompetenzgruppen wie folgt: **No-Code** ($S = 0$), **Low-Code** ($S = 1$ bis 5), **Mid-Code** ($S = 6$ bis 10) und **High-Code** ($S = 11$ bis 13). Diese Cutoffs werden

a priori festgelegt, um nachträgliche Anpassungen an die beobachteten Ergebnisse zu vermeiden.

Gruppe	Dreyfus-Level	Score	Charakteristik
No-Code	Novice	0	Kein mentales Modell von Programmierung. Verlässt sich zu 100% auf KI als Black Box.
Low-Code	Advanced Beginner	1–5	Kennt Grundkonzepte, aber keine Architekturmuster. Hohes Risiko für <i>Spaghetti-Code</i> und <i>Automation Bias</i> .
Mid-Code	Competent	6–10	Versteht Zusammenhänge, plant Schritte. Kann KI-Fehler meist erkennen (<i>System 2</i> Aktivierung möglich).
High-Code	Proficient/Expert	11–13	Intuitives Verständnis. Nutzt KI zur Beschleunigung von Boilerplate, nicht zur Lösungsfindung.

Tabelle 3.2: Mapping von Kompetenzgruppen auf das Dreyfus-Modell

Reduktion von Self-Report-Bias und Plausibilitätschecks. Da Teile der Operationalisierung auf Selbstauskunft beruhen, werden Likert-Items mit klaren Ankern formuliert (z. B. von „benötige Anleitung bei Debugging/Testfails“ bis „kann Fehler systematisch isolieren und Tests zielgerichtet einsetzen“). Zusätzlich werden Plausibilitätsregeln dokumentiert (z. B. auffällig hohe Selbsteinschätzung bei gleichzeitig 0 Jahren Erfahrung oder umgekehrt). Diese Checks dienen ausschließlich der Transparenz und markieren potenziell unsichere Einstufungen. Sie führen nicht zu einer nachträglichen, fallbezogenen „Korrektur“ einzelner Personen.

Optionale Sensitivitätsanalyse (Future Work). Als optionale Erweiterung (nicht Teil dieser Arbeit) könnte in zukünftiger Arbeit geprüft werden, wie stabil die beschriebenen Muster gegen plausible Alternativen der Kompetenz-Operationalisierung sind (z. B. leicht verschobene Cutoffs oder Variationen der Punktmappings der Selbsteinschätzungsitems). Solche Sensitivitätsläufe würden hier lediglich die Abhängigkeit der Interpretation von der gewählten Gruppendifinition transparent machen; es wird dabei keine bestimmte Richtung oder Stärke eines Effekts erwartet.

Threats to validity (Kompetenzmessung). Kompetenz ist ein latentes Konstrukt und kann durch Berufsjahre, Ausbildung und Selbsteinschätzung nur approximiert werden (Konstruktvalidität). Zudem kann Selbstauskunft systematisch verzerrt sein (Selbstüberschätzung/soziale Erwünschtheit), und Likert-Skalen können zwischen Personen unterschiedlich interpretiert werden (Messinvarianz). Schließlich ist allgemeine Programmierkompetenz nicht identisch mit domänenspezifischer Routine im verwendeten Stack (Konfundierung durch Technologie-Fit). Gegenmaßnahmen sind die *a priori* festgelegte, auditierbare Rubrik, die Ankerung der Selbsteinschätzung, dokumentierte Plausibilitäts-

checks sowie die als optionale Erweiterung vorgeschlagene Sensitivitätsanalyse gegen alternative Cutoffs/Mappings. Die Kompetenzgruppen bleiben damit eine interpretierbare Analyseperspektive, ohne Einzelfallbewertungen abzuleiten.

3.4 Kontrolle von Störvariablen (Confounder)

Um die interne Validität zu sichern, wurden folgende Störvariablen kontrolliert:

- **IDE-Familiarity:** Alle Teilnehmenden nutzten VS Code. Unterschiede in der Vertrautheit wurden im Pre-Survey erfasst.
- **Domain Knowledge:** Das Todo-App-Szenario ist generisch genug, um kein spezifisches Domänenwissen (wie z.B. in der Medizin) vorauszusetzen.
- **Englischkenntnisse:** Da Copilot primär auf englischem Code trainiert ist, wurden Grundkenntnisse vorausgesetzt.

3.5 Die Testumgebung als unabhängige Variable: Ecological Validity

Die Wahl einer **Brownfield-Umgebung** (bestehender Code) statt Greenfield (leeres Blatt) ist essenziell für die *ökologische Validität* der Studie. In der industriellen Praxis arbeiten Entwickler:innen zu ca. 90% in bestehenden Systemen (Brandtner u. a. 2024).

- **Kontext-Sensitivität (RAG-Test):** Das KI-Assistenzsystem muss den bestehenden Style (z. B. SCSS-Module, React Hooks) erkennen und adaptieren. Dies testet die Fähigkeit des Modells, Kontext zu nutzen, und die Fähigkeit des Nutzers, diesen Kontext bereitzustellen (z.B. durch Öffnen relevanter Dateien).
- **Kognitive Belastung:** Die Teilnehmenden müssen sich in der Struktur zurechtfinden, was für Low-Code-Teilnehmende eine zusätzliche Hürde darstellt (*Intrinsic Load*). Greenfield-Aufgaben (z.B. LeetCode) würden diese Komplexität ausblenden und die Ergebnisse verzerren.

Dies stellt sicher, dass wir *reale* Arbeitsbedingungen simulieren und nicht nur die Fähigkeit testen, isolierte Algorithmen zu generieren.

4 Durchführung und Methodik der Datenerhebung

Während das vorangegangene Kapitel das Forschungsdesign und die Hypothesen definiert hat, beschreibt dieses Kapitel den konkreten Ablauf der Studie sowie die Operationalisierung der Messgrößen. Es wird detailliert dargelegt, wie die Datenerhebung organisiert wurde, um die interne Validität der Ergebnisse sicherzustellen.

4.1 Ablauf der Untersuchung

Die Studie wurde als kontrolliertes Experiment durchgeführt. Jede Sitzung folgte einem strikten Protokoll, um Störfaktoren zu minimieren. Der Ablauf gliederte sich in fünf Phasen:

1. **Onboarding (10 Min.):** Die Teilnehmenden erhielten eine Einführung in das Szenario („Sie sind neuer Entwickler im Team...“) und die Entwicklungsumgebung. Es wurde sichergestellt, dass GitHub Copilot aktiviert und funktionsfähig war.
2. **Pre-Survey (5 Min.):** Erfassung der demografischen Daten und Selbsteinschätzung zur Kompetenzeinstufung (siehe Kapitel 3).
3. **Vibe-Coding-Challenge (60 Min.):** Die Kernphase der Untersuchung. Die Teilnehmenden bearbeiteten die Aufgaben selbstständig. Eingriffe durch den Versuchsleiter erfolgten nur bei technischen Problemen (z. B. Absturz der IDE), nicht jedoch bei inhaltlichen Fragen. **Begründung des Zeitlimits:** Die Begrenzung auf 60 Minuten dient der organisatorischen Machbarkeit und fungiert als Stressor. Zeitdruck erhöht die Wahrscheinlichkeit, dass Teilnehmende auf heuristische Strategien (*System 1*) zurückgreifen. Dies kann den *Automation Bias* verstärken und simuliert realistische Deadline-Situationen.
4. **Post-Survey (5 Min.):** Unmittelbar nach Abschluss der Bearbeitung füllten die Teilnehmenden den Fragebogen zur subjektiven Wahrnehmung aus (NASA-TLX, TAM).
5. **Debriefing (5 Min.):** Kurzes Abschlussgespräch, um offene Fragen zu klären und qualitatives Feedback einzuholen, das im Survey nicht abgebildet werden konnte.

4.2 Operationalisierung der Metriken

Um die Hypothesen zu prüfen, wurden die abstrakten Konzepte (Qualität, Effizienz, Interaktion) in messbare Indikatoren übersetzt.

4.2.1 Funktionale Korrektheit (Effectiveness)

Die funktionale Korrektheit wird binär pro Aufgabe bewertet:

- **Erfüllt (1):** Alle Akzeptanzkriterien („Done when“) sind erfüllt, Tests bestehen.
- **Nicht erfüllt (0):** Feature fehlt, ist fehlerhaft oder Tests schlagen fehl.

Die Summe der erfüllten Aufgaben ergibt den *Functional Score* (0–8).

4.2.2 Effizienz (Efficiency)

Da die Zeit auf 60 Minuten begrenzt war, wird Effizienz primär über den Fortschritt gemessen:

$$E = \frac{\text{Anzahl erfüllter Aufgaben}}{\text{Benötigte Zeit (min)}} \quad (4.1)$$

Für Teilnehmende, die alle Aufgaben vor Ablauf der Zeit lösten, steigt der Effizienzwert entsprechend.

4.2.3 Codequalität (Quality)

Die Qualität des erzeugten Codes wird anhand statischer Analyse und manuellem Review bewertet:

- **Linting-Fehler:** Anzahl der Warnungen und Fehler durch ESLint (objektiv).
- **Wartbarkeit:** Subjektive Bewertung der Lesbarkeit und Struktur (z. B. sinnvolle Variablennamen, Modularisierung) auf einer Likert-Skala (1–5). Zur Sicherung der Reliabilität orientiert sich die Bewertung an den Clean-Code-Prinzipien von Martin (2009).

4.2.4 Interaktionsqualität (Interaction Quality)

Die Interaktion mit dem KI-Assistenzsystem wird durch quantitative Telemetrie und qualitative Beobachtung erfasst:

- **Acceptance Rate:** Das Verhältnis von akzeptierten zu angezeigten Vorschlägen (Ghost Text). Eine hohe Rate bei Low-Code-Teilnehmenden deutet auf *Automation Bias* hin.
- **Prompt Iterations:** Wie oft muss ein Prompt umformuliert werden, bis das Ergebnis passt? Dies misst die Fähigkeit zum *Prompt Engineering*.

- **Backtracking:** Wie oft wird akzeptierter Code wieder gelöscht? Ein Indikator für *Trial & Error*.
- **Prompt-Typologie:** Unterscheidung zwischen *Inline-Kommentaren* (Code-nahe Steuerung) und *Chat-Panel* (konzeptionelle Fragen).

4.3 Güte der Untersuchung (Threats to Validity)

Die Validität der Studie wird anhand der Kategorien von Cook und Campbell (1979) diskutiert.

4.3.1 Konstruktvalidität (Construct Validity)

Messen wir wirklich das, was wir messen wollen?

- **Problem:** „Kompetenz“ ist schwer zu quantifizieren.
- **Mitigation:** Triangulation der Kompetenzmessung durch Selbsteinschätzung (Pre-Survey), objektive Berufsjahre und qualitative Beobachtung während der Aufgaben.

4.3.2 Interne Validität (Internal Validity)

Können die beobachteten Effekte eindeutig der unabhängigen Variable (Kompetenz) zugeschrieben werden?

- **Selection Bias:** Die Zuteilung zu den Gruppen basierte auf Selbsteinschätzung. Um dies zu mitigieren, wurde der Algorithmus zur Klassifizierung strikt angewendet.
- **Experimenter Bias (Rosenthal-Effekt):** Die Erwartungshaltung des Versuchsleiters könnte die Teilnehmenden beeinflussen. **Gegenmaßnahme:** Standardisierte Instruktionen und minimale Interaktion während der Coding-Phase.
- **Compensatory Rivalry (John Henry Effect):** Wenn eine Gruppe sich benachteiligt fühlt, könnte sie sich mehr anstrengen. Da alle Gruppen Copilot nutzten, ist dieses Risiko gering. Dennoch könnten High-Code-Teilnehmende versuchen, „besser als die KI“ zu sein (Saretsky 1972).

4.3.3 Externe Validität (External Validity)

Sind die Ergebnisse verallgemeinerbar?

- **Hawthorne-Effekt:** Das Wissen, beobachtet zu werden, verändert das Verhalten. Dies ist in Laborstudien unvermeidbar, wurde aber durch die realistische Aufgabenstellung (Flow-Erleben) abgemildert.
- **Ecological Validity:** Die Nutzung einer Brownfield-Umgebung erhöht die Übertragbarkeit auf Unternehmenskontexte im Vergleich zu synthetischen LeetCode-Aufgaben massiv.

4.3.4 Reliabilität (Reliability)

Sind die Ergebnisse reproduzierbar?

- **Inter-Rater Reliability:** Die Bewertung der Codequalität (Wartbarkeit) unterliegt einer gewissen Subjektivität. Durch die Verwendung eines standardisierten Kriterienkatalogs (basierend auf ISO 25010) wird versucht, die Varianz zu minimieren.

4.4 Datenanalyse-Strategie

Die Auswertung erfolgt mittels eines *Mixed-Methods*-Ansatzes:

1. **Quantitative Analyse:** Vergleich der Mittelwerte von Functional Score und Effizienz zwischen den vier Gruppen. Aufgrund der kleinen Stichprobe werden deskriptive Statistiken und Visualisierungen (Boxplots) bevorzugt.
2. **Qualitative Analyse:** Inhaltsanalyse der Screen-Recordings und des Codes, um Verhaltensmuster zu identifizieren (z. B. „Trial & Error“ bei Low-Code-Teilnehmenden vs. „Review & Refine“ bei High-Code-Teilnehmenden).

Dieser methodische Aufbau stellt sicher, dass nicht nur das *Ergebnis* (Code), sondern auch der *Prozess* (Interaktion) beleuchtet wird.

5 Technische Realisierung der Testumgebung

Dieses Kapitel dokumentiert die technische Implementierung der *BuyIn Todo Sandbox*, die als experimentelle Plattform für diese Studie dient. Es erläutert, wie die Brownfield-Umgebung architektonisch aufgebaut ist, um realistische Unternehmensanforderungen zu simulieren. Zudem wird beschrieben, welche technischen Maßnahmen ergriffen wurden, um eine reproduzierbare Datenerhebung zu gewährleisten.

5.1 Anforderungen an die Sandbox

Um valide Aussagen über Vibe-Coding in Unternehmenskontexten treffen zu können, musste die Testumgebung folgende Anforderungen erfüllen:

1. **Realismus (Brownfield):** Die Codebasis darf nicht leer sein („Greenfield“), sondern muss bestehende Strukturen, technische Schulden und Konventionen enthalten, die das KI-Assistenzsystem und der Mensch verstehen müssen.
2. **Komplexität:** Der Tech-Stack muss modern, aber komplex genug sein, um triviale Lösungen zu verhindern.
3. **Isolierbarkeit:** Die Umgebung muss für jeden Teilnehmenden identisch zurückgesetzt werden können (Containerisierung).
4. **Messbarkeit:** Das System muss automatisierte Tests bereitstellen, um den funktionalen Fortschritt objektiv zu messen.

5.2 Architektur und Tech-Stack: Kognitive Dimensionen

Die Sandbox ist als klassische Full-Stack-Webanwendung konzipiert. Die Architekturwahl hat direkte Auswirkungen auf die kognitive Belastung der Teilnehmenden, analysiert durch das Framework der *Cognitive Dimensions of Notation* (Green 1989).

5.2.1 Backend: Node.js & Express (Layered Architecture)

Das Backend basiert auf Node.js mit dem Express-Framework und folgt einer strikten Schichtenarchitektur:

- **Routes:** Definition der API-Endpunkte.
- **Controllers:** Verarbeitung der HTTP-Requests und Validierung.
- **Services:** Kapselung der Geschäftslogik.
- **Repositories:** Datenzugriffsschicht.

Kognitive Dimension - Viscosity (Zähigkeit): Diese Trennung erhöht die *Viscosity* für Novizen, da eine Änderung an mehreren Stellen (Datei-Hopping) durchgeführt werden muss. Für Copilot hingegen ist diese Struktur vorteilhaft. Sie zieht klare semantische Grenzen, die das KI-Assistenzsystem über Dateinamen und Pfadkontext nutzen kann.

5.2.2 Frontend: React & Vite

Das Frontend nutzt React mit TypeScript. Die Komponenten sind funktional aufgebaut und nutzen Hooks. **Kognitive Dimension - Hidden Dependencies:** Die Verwendung von *Custom Hooks* und *Context API* erzeugt *Hidden Dependencies*. Low-Code-Teilnehmende sehen im UI-Code nicht sofort, woher die Daten kommen. Dies testet die Fähigkeit des KI-Assistenzsystems, diese Abhängigkeiten aufzulösen, sowie die Fähigkeit des Nutzers, dem KI-Assistenzsystem den passenden Kontext zu geben.

5.3 Technische Funktionsweise von Copilot in der Sandbox

Um zu verstehen, warum die Sandbox Copilot fordert, muss die technische Funktionsweise betrachtet werden.

5.3.1 Prompt Construction und Context Retrieval

Copilot sendet nicht nur den Code vor dem Cursor an das Modell. Der *Copilot Proxy* in der IDE sammelt Kontext aus:

- **Jaccard Similarity:** Offene Tabs werden nach Ähnlichkeit zum aktuellen Code gescannt.
- **Semantic Search:** Vektorisierte Suche im Projekt (bei neueren Versionen).

In der Sandbox sind die Aufgaben so gestaltet, dass die Lösung oft in einer *anderen* Datei liegt (z.B. muss der Controller angepasst werden, um den Service zu nutzen). Wenn der Nutzer diese Datei nicht öffnet, fehlt dem KI-Assistenzsystem der Kontext. Dadurch wird indirekt die Kompetenz des Nutzers gemessen, dem KI-Assistenzsystem die richtigen „Hinweise“ zu geben.

5.3.2 Token Prediction und Ranking

Das Modell generiert mehrere Vorschläge (Beams) und rankt diese nach Wahrscheinlichkeit. In einer Brownfield-Umgebung ist die Wahrscheinlichkeit für „korrekten“ Code höher, wenn er dem Stil der Umgebung entspricht. High-Code-Teilnehmende erkennen, ob ein Vorschlag den Stil bricht (z.B. `var` statt `const`); Low-Code-Teilnehmende häufig nicht.

5.4 Design der Aufgaben (Workload Grading)

Die Aufgaben sind so gestaffelt, dass sie die kognitive Last sukzessive erhöhen:

Task	Beschreibung	Kognitive Anforderung
1	Persistenz (JSON)	Verstehen des Repository-Patterns (Backend).
2	User-Accounts	Einführung neuer Entitäten, Anpassung mehrerer Layer.
3	Kategorien (UI)	Frontend-Logik, State-Management.
4	Attachments	Umgang mit Binärdaten, API-Erweiterung.
5	Kalender	Komplexe Logik, Datumsberechnung (hoher Intrinsic Load).
6	Email-Verifikation	Integration externer Konzepte (Mocking).
7	Performance	Optimierung, kein funktionales Feature (Experten-Task).
8	ICS-Export	Standard-Format, reines Mapping (Erholung).

Tabelle 5.1: Aufgaben-Design und kognitive Anforderungen

5.5 Automatisierung der Evaluation

Um die Auswertung der Ergebnisse zu standardisieren, wurde ein Evaluations-Framework entwickelt.

5.5.1 Das Analyse-Skript (`run.js`)

Ein Node.js-Skript automatisiert die Prüfung der Lösungen. Es führt folgende Schritte aus:

1. **Test-Execution:** Ausführung der Unit-Tests (Vitest) und End-to-End-Tests (Playwright).
2. **Linting:** Prüfung auf Code-Style-Verletzungen mittels ESLint.
3. **Report-Generierung:** Zusammenfassung der Ergebnisse in einer JSON-Datei.

5.5.2 Grenzen der LLM-gestützten Code-Analyse

Zusätzlich zur rein funktionalen Prüfung wird ein LLM-Prompt-Template verwendet, um qualitative Aspekte des Codes zu bewerten. Hierbei muss kritisch angemerkt werden, dass LLMs zu *Halluzinationen* neigen können: Sie finden Fehler, die nicht existieren, oder übersehen echte Probleme. Der Score dient daher nur als Indikator und wird stichprobenartig manuell validiert.

5.6 Relevanz für den Unternehmenskontext (BuyIn)

Die gewählte Architektur und Methodik spiegelt reale Herausforderungen bei BuyIn wider:

- **Legacy-Integration:** Wie bei BuyIn müssen Entwickler oft in gewachsenen Strukturen arbeiten, die nicht perfekt dokumentiert sind. Vibe-Coding kann hier helfen, den „impliziten Kontext“ schneller zu erschließen.
- **Security & Compliance:** Der Einsatz von Copilot in Enterprise-Umgebungen birgt Risiken (Datenabfluss, Lizenzverletzungen). Die Sandbox simuliert dies durch Aufgaben, bei denen sensible Daten (User-Accounts) behandelt werden müssen.
- **Skalierbarkeit:** Die modulare Struktur der Sandbox zeigt, wie Copilot in größeren Teams (wo Modularisierung Pflicht ist) performt, im Gegensatz zu kleinen Skript-Projekten.

5.7 Containerisierung und Reproduzierbarkeit

Die gesamte Umgebung ist mittels Docker containerisiert. Dies stellt sicher, dass alle Teilnehmenden – unabhängig von ihrem Betriebssystem – unter identischen Bedingungen arbeiten und technische Störfaktoren (z.B. unterschiedliche Node-Versionen) eliminiert werden.

5.7.1 Reproduzierbarkeit / Artefaktzugang

Die in dieser Arbeit berichteten quantitativen Ergebnisse (Tests, Linting und daraus abgeleitete Scores) sind aus dem Repository heraus nachvollziehbar reproduzierbar. Neben der containerisierten Sandbox liegen die zentralen Auswerteskripte sowie die genutzten Rohdaten (Chat- und Survey-Exporte) versioniert vor.

Ablage der relevanten Artefakte.

- `backend/`, `frontend/`: Anwendungscode inkl. Tests und ESLint-Konfiguration.
- `evaluation/`: Ausführbare Evaluationsskripte und erzeugte Ergebnisdateien (z.B. `result.json`, `result.md`).

- `thesis/chats/`: exportierte Chatverläufe mit dem KI-Assistenzsystem (`.docx`).
- `thesis/surveys/`: Survey-Export zur Kompetenz-Operationalisierung (`.xlsx`).

Minimaler Reproduktionspfad (4 Schritte).

1. Abhängigkeiten installieren: `npm ci -prefix backend`
2. Abhängigkeiten installieren: `npm ci -prefix frontend`
3. Evaluation ausführen: `node evaluation/run.js`
4. Ergebnisse einsehen: `evaluation/result.json`, `evaluation/result.md`
(zusätzlich Detailreports in `backend/test-results.json`, `frontend/test-results.json`, `frontend/playwright-report.json` sowie `backend/lint-results.json`, `frontend/lint-results.json`).

Erweiterte Reproduktion (optional). Für die im Ergebnisteil verwendete Pipeline-Klassifikation („Attempted“ aus Chat-Signalen, „Implemented_{static}“ aus Code-Heuristiken, „Verified“ durch Testausführung) kann zusätzlich `node evaluation/run_pipeline_v3.js` ausgeführt werden. Das Skript erzeugt u.a. `evaluation/task_pipeline_v3.json` und `evaluation/task_pipeline_v3.csv`. Die Survey-Zuordnung zu Branches liegt als erzeugtes Artefakt vor (`evaluation/survey_mapping.json`); die Neugenerierung via `node evaluation/map_survey.js` ist möglich, erfordert jedoch ggf. eine Anpassung der im Skript hinterlegten Dateipfade.

Artefakt	Zweck	Erzeugt durch
<code>evaluation/result.json</code>	Aggregierte Task- und Lint-Ergebnisse pro Branch/Run	<code>run.js</code>
<code>evaluation/result.md</code>	Menschlich lesbare Zusammenfassung der Checks	<code>run.js</code>
<code>backend/test-results.json</code>	Backend-Testergebnisse (Jest) als JSON-Report	<code>run.js</code>
<code>frontend/test-results.json</code>	Frontend-Testergebnisse (Vitest) als JSON-Report	<code>run.js</code>
<code>frontend/playwright-report.json</code>	E2E-Testergebnisse (Playwright) als JSON-Report	<code>run.js</code>
<code>evaluation/task_pipeline_v3.json</code>	Pipeline-Klassifikation je Branch (Attempted/Implemented/Verified)	<code>run_pipeline_v3.js</code>
<code>evaluation/survey_mapping.json</code>	Zuordnung: Branch – Teilnehmende Person – Kompetenzgruppe	<code>map_survey.js</code>

Tabelle 5.2: Zentrale Artefakte und ihre Erzeugung im Repository

6 Ergebnisse

Zur Orientierung gilt: Die Pipeline-Status *Attempted*, *Implemented_{static}* und *Verified* beschreiben unterschiedliche Evidenzebenen desselben Arbeitsprozesses und sind keine Qualitäts- oder Kompetenzbewertung. Insbesondere gilt: *Verified* ist nicht gleich „besserer Entwickler“. *Implemented_{static}* meint strukturelle Artefakte in der Codebasis; *Verified* ist strikt testgebunden.

6.1 Datengrundlage und Toolkontext

Dieses Unterkapitel beschreibt die Datengrundlage der Untersuchung sowie den technischen Kontext der durchgeführten *Vibe-Coding-Challenge*. Die Analyse basiert auf einem Datensatz von $N = 18$ Probanden, die im Rahmen einer kontrollierten Coding-Challenge Aufgaben zur Entwicklung einer Todo-Applikation bearbeiteten.

6.1.1 Studiensetting und Rahmenbedingungen

Die Studie wurde unter einheitlichen Rahmenbedingungen durchgeführt, um die Vergleichbarkeit der Ergebnisse zu gewährleisten:

- **Zeitlimit:** Alle Probanden hatten ein striktes Zeitlimit von 60 Minuten für die Bearbeitung der Aufgaben.
- **Tooling:** Als Entwicklungsumgebung diente Visual Studio Code mit der GitHub Copilot Extension.
- **KI-Modell:** Als zugrundeliegendes Large Language Model (LLM) wurde **Claude Sonnet 4.5** verwendet. Die Interaktion erfolgte über das Chat-Interface von GitHub Copilot, wobei das Modell explizit als Backend konfiguriert war.
- **Aufgabenstellung:** Die Probanden erhielten eine Liste von 8 Aufgaben (T1–T8) mit steigender Komplexität, beginnend bei Backend-Persistenz bis hin zu komplexen Frontend-Features und Export-Funktionen.

6.1.2 Datenquellen und Artefakte

Die empirische Auswertung stützt sich auf drei primäre Datenquellen, die eine Triangulation der Ergebnisse ermöglichen:

1. **Code-Artefakte (Git-Repositories):** Die finalen Implementierungen der Probanden liegen als separate Git-Branche vor. Diese Branches enthalten den vollständigen Quellcode zum Zeitpunkt des Zeitablaufs.
 - Pfad: Remote-Branche mit dem Suffix `*-vibe` (z.B. `origin/nachname-vorname-vibe`).
2. **Chat-Logs:** Die vollständigen Interaktionsprotokolle zwischen Proband und KI-Assistenzsystem wurden exportiert. Diese Protokolle enthalten alle Prompts der Nutzer sowie die Antworten und Code-Generierungen der KI.
 - Pfad: `thesis/chats/` (Format: `.docx`).
3. **Survey-Daten:** Im Anschluss an die Challenge füllten die Probanden einen Fragebogen zur Selbsteinschätzung und subjektiven Wahrnehmung der KI-Interaktion aus.
 - Pfad: `thesis/surveys/Probanden-Einstufungsformular (Responses).xlsx`.

6.1.3 Dateninventar und Vollständigkeit

Tabelle 6.1 zeigt das Inventar der verfügbaren Datenartefakte. Insgesamt liegen für 18 Probanden Daten vor.

Artefakt-Typ	Anzahl	Anmerkung
Git-Branche (<code>*-vibe</code>)	18	Vollständig als Remote-Branche vorhanden
Chat-Logs	18	Vollständig für alle Teilnehmer
Survey-Einträge	18	Vollständig ($N = 18$)

Tabelle 6.1: Inventar der Datenartefakte.

6.1.4 Zuordnung und Zusammenführung der Daten

Die Zusammenführung der Datenquellen erfolgte über eine Mapping-Tabelle, die die Klarnamen aus dem Survey und den Chat-Dateinamen den entsprechenden Git-Branche zuordnet.

- **Survey ↔ Branch:** Das Survey-Feld "Branch-Name" sowie der Name des Probanden dienten als Schlüssel.
- **Chat ↔ Branch:** Die Dateinamen der Chat-Logs (z.B. `Nachname_Vorname.docx`) wurden normalisiert und den entsprechenden Branches (z.B. `nachname-vorname-vibe`) zugeordnet.

Die Survey-Daten enthalten zudem eine Einteilung in Kompetenzgruppen (No-Code, Low-Code, Mid-Code, High-Code), die in der späteren Analyse als vergleichendes Merkmal herangezogen wird.

6.1.5 Reproduzierbarkeit und Auswertungswerkzeuge

Um die Ergebnisse objektiv und reproduzierbar zu gestalten, kommen automatisierte Analysewerkzeuge zum Einsatz, deren Resultate im Ordner `evaluation/` persistiert werden:

- **Statische Code-Analyse:** Einsatz von `ESLint` zur Überprüfung der Code-Qualität und Einhaltung von Standards.
- **Automatisierte Tests:** Ausführung der vorhandenen Test-Suite (basierend auf `Jest/Vitest`), um die funktionale Korrektheit der Implementierungen (Task Completion) zu verifizieren.
- **Chat-Parsing:** Algorithmische Auswertung der Chat-Logs zur Bestimmung von `AttemptedTasks` und Identifikation von Prompting-Mustern.

6.1.6 Abgrenzung zu Kapitel 7

Das vorliegende Kapitel 6 beschränkt sich auf die deskriptive Darstellung der Ergebnisse und die objektive Auswertung der Daten. Eine Interpretation dieser Ergebnisse, die Diskussion von Ursache-Wirkungs-Beziehungen sowie die Beantwortung der Forschungsfragen erfolgen im anschließenden Kapitel 7 (Diskussion).

6.2 Task-Pipeline-Analyse

In diesem Abschnitt wird der Fortschritt der Probanden entlang der Aufgaben-Pipeline (T1–T5) dargestellt. Ziel ist es, den *Drop-off* über die Aufgaben hinweg sichtbar zu machen und zwischen Intention (Chat), Umsetzung (Code) und testbasierter Bestätigung (Integrationstests) zu unterscheiden.

6.2.1 Methodische Definitionen

Zur Einordnung werden drei Statuskategorien verwendet:

- **Attempted (Versucht):** Ein Task gilt als *Attempted*, wenn er im Chat-Verlauf explizit als Ziel, Anforderung oder nächster Arbeitsschritt formuliert wurde. Die bloße Existenz von Dateien ohne entsprechenden Kontext im Chat genügt nicht.
- **Implemented_{static} (Umgesetzt, Artefakt/Heuristik):** Ein Task gilt als *Implemented_{static}*, wenn er in den Auswertungsartefakten als implementiert markiert ist (heuristische, statische Indikatoren; z. B. Feld `implemented` in `evaluation/task_pipeline_v3.json` bzw. Implementationsklassen in `evaluation/deep_code_analysis.json`).
- **Verified (Testbasiert bestätigt):** Ein Task gilt als *Verified*, wenn die zugehörigen automatisierten Integrationstests erfolgreich durchlaufen wurden (Feld `verified` in `evaluation/task_pipeline_v3.json`).

Hierarchie der Metriken (Trichterdarstellung). Für die Trichterdarstellung wird in Kapitel 6 eine abgeleitete Evidenzstufe *Implemented** verwendet:

$$\textit{Implemented}^* := \textit{Attempted} \wedge (\textit{Implemented}_{\textit{static}} \vee \textit{Verified}).$$

Damit gilt per Konstruktion $\textit{Verified} \subseteq \textit{Implemented}^*$ (ein bestandener taskspezifischer Test wird als hinreichender Implementationsnachweis auf Verhaltensebene gewertet). *Verified* ist jedoch nicht zwingend ein Subset von *Implemented_{static}*: *Implemented_{static}* basiert auf heuristischen, strukturellen Erkennungsregeln und kann funktionsfähige Implementationen übersehen (z. B. abweichende Benennungen, alternative Architekturpfade, oder Implementationen außerhalb der erwarteten Muster). *Verified* ist damit kein Qualitätsurteil, sondern ein testgebundenes Diagnose-Signal; *Implemented_{static}* ist ein Artefakt-Signal.

Hinweis zur Testmethodik (V2). Für die Pipeline-Auswertung wurden die Tests (insbesondere T2, sowie in Teilen T3–T5) als API-Integrationstests (*Supertest* gegen Express) gestaltet, um weniger abhängig von internen Architekturentscheidungen (z.B. Service-/Repository-Struktur) zu sein. *Verified* bedeutet in dieser Studie daher explizit: „besteht die aktuellen Tests“ – nicht „fachlich vollständig korrekt gelöst“. *Verified* darf folglich nie isoliert interpretiert werden, sondern immer im Verhältnis zu *Attempted* und *Implemented**.

6.2.2 Pipeline-Ergebnisse (T1–T5)

Tabelle 6.2 fasst den Status der Aufgaben T1 bis T5 für alle $N = 18$ Probanden zusammen. Der Fokus liegt aufgrund des strikten Zeitlimits (60 Minuten) auf diesen frühen Aufgaben.

Task	Attempted (<i>n</i>)	Implemented* (<i>n</i>)	Verified (<i>n</i>)
T1: Persistenz	18 (100%)	17 (94%)	10 (56%)
T2: Auth	18 (100%)	17 (94%)	4 (22%)
T3: Kategorien	15 (83%)	13 (72%)	13 (72%)
T4: Attachments	13 (72%)	8 (44%)	0 (0%)
T5: Kalender	13 (72%)	0 (0%)	0 (0%)

Tabelle 6.2: Task-Pipeline: Von der Intention zur verifizierten Lösung ($N = 18$). *Attempted* stammt aus Chat-Signalen; *Verified* aus V3-Integrationstests; *Implemented** ist die Trichter-Evidenzstufe $\textit{Attempted} \wedge (\textit{Implemented}_{\textit{static}} \vee \textit{Verified})$.

6.2.3 Beobachtungen und Interpretation

Für T3–T5 ist der *Drop-off* in erster Linie zeitgetrieben: Unter dem 60-Minuten-Limit werden Integrationsaufgaben häufiger begonnen (*Attempted*), aber seltener konsistent bis zur Evidenzstufe *Implemented** geführt und noch seltener testkonform stabilisiert

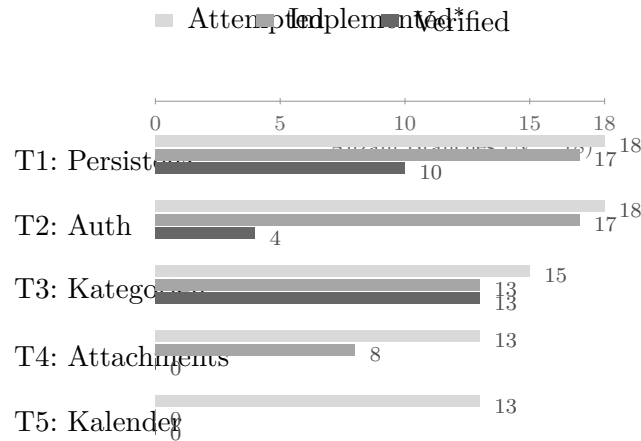


Abbildung 6.1: Trichterdarstellung der Pipeline-Ergebnisse für T1–T5 ($N = 18$). Pro Task zeigt die Grafik die Anzahl der Branches mit *Attempted* (Chat-Signal), *Implemented** ($Attempted \wedge (Implemented_{static} \vee Verified)$) sowie *Verified* (bestandene V3-Integrationstests). Die Werte entsprechen Tabelle 6.2.

(*Verified*). „Nicht implementiert“ bedeutet dabei nicht „nicht verstanden“, sondern häufig „nicht mehr rechtzeitig integriert, refaktoriert oder debuggt“.

T1 (Persistenz). Alle Probanden adressierten Persistenz im Chat (100%), und die hohe Implementierungsrate (94%) zeigt, dass Persistenz als Kernanforderung verstanden wurde. Die Verifikationsrate (56%) weist darauf hin, dass eine robuste, testkonforme Persistenz (z.B. stabiler CRUD-Fluss, konsistente IDs, fehlerfreie Serialisierung) innerhalb des Zeitfensters nicht in allen Fällen erreicht wurde.

T2 (User Accounts/Auth). Auch T2 wurde von allen Probanden versucht (100%) und sehr häufig implementiert (94%). Die vergleichsweise niedrige *Verified*-Rate (22%) ist weniger als Qualitätsurteil zu lesen, sondern als Diagnose-Signal: Auth-Implementierungen sind stark architektur- und konfigurationsabhängig (Endpunktpfade, Token- vs. Cookie-Mechanismen, Zugriffskontrollen), wodurch selbst funktionierende Lösungen an den konkreten Testannahmen scheitern können.

T3 (Kategorien). Der Drop-off (83% *Attempted* vs. 72% *Implemented**) deutet darauf hin, dass Kategorien häufig begonnen werden, aber nicht in allen Fällen bis zur stabilen, wiederholbaren Integration geführt werden. T3 illustriert zudem die begriffliche Trennung: In `evaluation/task_pipeline_v3.json` ist T3 in 7/18 Branches als `implemented` ($Implemented_{static}$) markiert, aber in 13/18 Branches `verified`. Für die Trichterdarstellung wird dies über *Implemented** konsistent zusammengeführt (T3: 13/18), ohne daraus eine Rangordnung der Signale abzuleiten.

T4 (Attachments). Trotz 72% *Attempted* und 44% *Implemented** liegt *Verified* bei 0%. Das spricht für hohe Varianz in der Umsetzung (unterschiedliche Upload-Mechanismen, Feldnamen, Storage-Strategien) und dafür, dass die Testannahmen die Heterogenität nicht ausreichend abdecken. In diesem Fall ist *Verified* vor allem als Hinweis auf Test-/Schnittstelleninkonsistenz zu lesen, nicht als Beleg fehlender Kompetenz.

T5 (Kalender/Multi-Day). T5 wurde häufig begonnen (72% *Attempted*), jedoch liegt

im (V3-)Pipeline-Lauf kein reproduzierbares, taskspezifisches API-Verhalten vor, das die Kalender-/Multi-Day-Semantik zuverlässig bestätigt (0% *Verified*). Entsprechend bleibt *Implemented** hier bei 0%: Weder konnte taskspezifische Logik statisch identifiziert werden, noch wurde sie durch konsistente Tests bestätigt. Inhaltlich ist dies mit dem Zeitlimit vereinbar: Kalenderlogik erfordert typischerweise zusätzliche Felder (Range), Validierung und Query-Semantik (Overlap), die unter 60 Minuten häufig nicht mehr end-to-end stabilisiert werden.

Zwischenfazit. Insgesamt zeigt die Pipeline den erwartbaren Trichter: Alle Teilnehmer starten stark bei T1/T2, während komplexere Integrationsaufgaben (T3–T5) unter Zeitdruck seltener vollständig umgesetzt werden. *Verified*-Werte werden in dieser Arbeit nicht „gefeiert“, sondern als Diagnose-Signal genutzt – für Testqualität, Architekturdiversität sowie die Differenz zwischen beobachtbarem Verhalten und tatsächlich implementierter Fachlogik.

Methodischer Hinweis zur Interpretation

Die Pipeline dient der Strukturanalyse von Arbeitsverläufen (Starten, Umsetzen, Stabilisieren) und damit der Beschreibung typischer Engpässe unter Zeitdruck – nicht der Bewertung individueller Leistung.

6.3 Detaillierte Lösungsstrategien pro Task (T1–T5)

Dieses Unterkapitel beschreibt typische Lösungsstrategien, die in den $N = 18$ *-vibe-Banches für die Tasks T1–T5 beobachtet wurden. Grundlage sind Repository-Struktur und Implementationsartefakte (Backend/Frontend, Datenhaltung), Chat-basierte *Attempted*-Signale sowie Survey-Angaben (aggregiert, ohne Klarnamen). Die Darstellung ist rein deskriptiv.

Einordnung der Datenbasis (kurz)

T1 und T2 wurden im Chat durchgängig adressiert (18/18), T3 in 15/18 sowie T4/T5 in jeweils 13/18 Fällen. Die Konfidenz sinkt dabei von T1/T2 zu T4/T5. In den Surveys werden Zeitdruck und Debugging am häufigsten als Hürden genannt (je 5/18), gefolgt von Aufgabenverständnis (3/18). Die Selbsteinstufung verteilt sich über No-Code (5), Low-Code (4), Mid-Code (5) und High-Code (4).

T1 – Persistente Todos

1. **Ziel des Tasks (kurz, technisch).** Persistente Speicherung von Todos über Prozess-Neustarts hinweg, inkl. stabilem CRUD-Verhalten (Erstellen, Lesen, Aktualisieren, Löschen) und konsistenter Identifikatoren.
2. **Typische Implementierungsstrategien (mit Häufigkeiten).** Die dominante Strategie war SQLite-basierte Persistenz (14/18). Daneben traten MongoDB (1/18)

und PostgreSQL (1/18) sowie verbleibende Varianten wie In-Memory/sonstige Ablagen (2/18) auf. In der Code-Struktur zeigt sich häufig ein Repository-Pattern (z.B. `TodoRepository`) mit einer klaren Abgrenzung zwischen Controller/Route und Datenzugriff.

3. **Qualitätsunterschiede (robust vs. fragil).** Robust sind konsistente ID-Schemata und deterministisches CRUD. Fragil sind v.a. uneinheitliche ID-Felder (`id` vs. `_id`) und inkonsistente Update-Semantik.
4. **Typische KI-generierte Muster.** Häufig sind Repository-/Controller-Boilerplates und generische CRUD-Endpunkte, bei denen Randfälle (Validierung, Fehlercodes, ID-Normalisierung) nur teilweise konsistent umgesetzt sind.
5. **Warum der Task häufig abgebrochen oder nur teilweise umgesetzt wurde.** T1 wurde selten abgebrochen, aber häufig nur bis zu einem Minimal-CRUD stabilisiert. Zeitdruck und Debugging (DB-Setup/Build) begrenzen die Absicherung.

T2 – Benutzerkonten / Authentifizierung

1. **Ziel des Tasks (kurz, technisch).** Nutzerregistrierung und Login mit Zugriffskontrolle auf geschützte Endpunkte (z.B. `GET /api/todos`) sowie konsistenter Authentifizierungsweitergabe (Token oder Session).
2. **Typische Implementierungsstrategien (mit Häufigkeiten).** JWT ist dominant (11/18), gefolgt von Session/Cookie (4/18) und Hybrid-Ansätzen (1/18). Selten sind Sonderfälle (z.B. Hashing ohne klaren Token-Flow: 1/18) bzw. fehlende oder unklare Signale (1/18). Häufige Strukturmuster sind zentrale Auth-Middleware oder Prüfungen direkt im Controller.
3. **Qualitätsunterschiede (robust vs. fragil).** Robust sind eindeutige Token-/Session-Flows und konsistente Route-Guards. Fragil sind inkonsistente Header-/Cookie-Nutzung und divergierende Erwartungen zwischen Frontend und Backend.
4. **Typische KI-generierte Muster.** Boilerplate-Flows (Register/Login/Protect) treten häufig auf, teils mit Over-Engineering (Rollen/Claims) oder mit Minimalismus (fehlende Validierung/Expiry). Verbreitet sind Tutorial-nahe Copy-Paste-Muster (JWT + `bcrypt`).
5. **Warum der Task häufig abgebrochen oder nur teilweise umgesetzt wurde.** T2 ist stark schnittstellen- und konfigurationssensitiv (Endpunkte, Token-Feldnamen, Cookie-Flags) und wird laut Survey häufig durch Zeitdruck und Debugging begrenzt. Die testnahe Absicherung bleibt oft nachrangig.

T3 – Kategorien

1. **Ziel des Tasks (kurz, technisch).** Erweiterung des Todo-Datenmodells um Kategorisierung und Bereitstellung über API und UI, sodass Todos kategoriebasiert erstellt, gespeichert und angezeigt werden können.
2. **Typische Implementierungsstrategien (mit Häufigkeiten).** Ein separates Kategorie-Konzept (Model/Tabelle/Collection mit Relation via `categoryId`) dominiert (14/18). Inline-Kategorien als String sind selten (1/18). In 3/18 Branches ist das Kategorie-Konzept in Struktursignalen nicht klar erkennbar.
3. **Qualitätsunterschiede (robust vs. fragil).** Robust sind konsistente Persistenz und stabile Referenzen (IDs/Relation). Fragil sind rein UI-seitige Kategorien oder inkonsistente Relation/Serialisierung.
4. **Typische KI-generierte Muster.** Häufig sind zusätzliche `Category`-Modelle und Routen (teils mit unnötiger CRUD-Komplexität) sowie „Tags vs. Categories“-Mischformen, bei denen Begriffe variieren, die Datenrepräsentation aber nicht konsistent vereinheitlicht wird.
5. **Warum der Task häufig abgebrochen oder nur teilweise umgesetzt wurde.** T3 erhöht den Integrationsaufwand (Schema, UI, Backward-Compatibility). Unter Zeitdruck bleibt häufig die End-to-End-Stabilisierung (Persistenz/Anzeige/Tests) unvollständig.

T4 – Dateianhänge (Attachments)

1. **Ziel des Tasks (kurz, technisch).** Upload und Zuordnung von Dateien zu Todos (typischerweise `multipart/form-data`), Speicherung (Dateisystem oder DB) und Rückgabe verwertbarer Metadaten über die API.
2. **Typische Implementierungsstrategien (mit Häufigkeiten).** In 8/18 Branches ist ein Multer-/Upload-Ansatz erkennbar. In 10/18 Branches sind Attachments nicht oder nur indirekt oder unklar umgesetzt. Wo Upload existiert, ist Storage häufig lokal (Uploads-Ordner) oder metadatenbasiert (Dateiname/URL).
3. **Qualitätsunterschiede (robust vs. fragil).** Robust sind klare Upload-Verträge (Feldnamen/Metadaten) und stabile Zuordnung zum Todo. Fragil sind uneinheitliche Response-Formate und fehlende Metadaten-Persistenz.
4. **Typische KI-generierte Muster.** Multer-Templates mit Boilerplate-Routen sind häufig. Daneben treten Over-Engineering (komplexe Attachment-Entitäten) oder Minimal-Uploads ohne stabilen Metadaten-Rückkanal auf.
5. **Warum der Task häufig abgebrochen oder nur teilweise umgesetzt wurde.** T4 bündelt viele Fehlerquellen (`multipart`, Storage, Pfade/URLs, Frontend-Integration) und wird in Surveys häufig als Debugging-getrieben beschrieben. Unter Zeitdruck wird es oft hinter Persistenz/Auth priorisiert.

T5 – Kalender / Multi-Day

1. **Ziel des Tasks (kurz, technisch).** Abbildung von Todos im Kalender, inklusive Unterstützung von Zeiträumen (Multi-Day), sowie konsistente Query-/Filter-Semantik (z.B. Overlap-Logik) zur effizienten Darstellung in Kalenderansichten.
2. **Typische Implementierungsstrategien (mit Häufigkeiten).** In den Branches sind Range-Felder als Struktursignale verbreitet (z.B. `dueEndDate` neben `dueDate`). Gleichzeitig zeigen die Integrationstests (V3) für T5 keine reproduzierbare End-to-End-Semantik (0/18 verifiziert), was auf eine häufige Teilstrategie hindeutet: Felder/Komponenten werden vorbereitet, die vollständige Backend-Semantik (Overlap/Filter, stabiler API-Vertrag) bleibt jedoch unvollständig.
3. **Qualitätsunterschiede (robust vs. fragil).** Robust wäre Range-Validierung (Ende > Start), zuverlässige Persistenz und klare Overlap-Filterung. Fragil sind reine Feld-Präsenz ohne konsistente Query-/Filter-Semantik.
4. **Typische KI-generierte Muster.** Häufig sind UI-/Modell-Boilerplates (zusätzliche Datumsfelder, Kalender-Komponenten) sowie Copy-Paste von Range-Utilities ohne konsistenten API-Vertrag (Parameter/Overlap-Regel).
5. **Warum der Task häufig abgebrochen oder nur teilweise umgesetzt wurde.** T5 ist stark gekoppelt (Modell → Persistenz → Query → UI) und wird laut Chat/Survey häufig durch Zeitdruck und Debugging limitiert. Unter 60 Minuten bleibt die testbare Semantik (Overlap/Filter, konsistente API) oft unvollständig.

6.4 Code-Qualität und technische Metriken

Dieses Unterkapitel beschreibt beobachtbare Aspekte der Code-Qualität über alle 18 *-vibe-Banches hinweg. Im Fokus stehen technische Messwerte und Artefakte (Linting, Typisierungs-Surrogate, Teststabilität und Strukturindikatoren), die die Wartbarkeit und Robustheit des Systems ergänzend zur rein funktionalen Korrektheit beschreiben. Die Darstellung bleibt bewusst deskriptiv. Sie bildet Messstände und Auswertungsergebnisse ab, ohne Ursachen oder Wirkzusammenhänge abzuleiten. Als Datenbasis dienen ausschließlich die Auswertungsartefakte `evaluation/code_quality_metrics.json`, `evaluation/data_safe/lint_results.json`, `evaluation/data_safe/test_results.json`, `evaluation/deep_code_analysis.json` und `evaluation/task_pipeline_v3.json`.

6.4.1 ESLint und Typisierung

Die ESLint-Ergebnisse liegen als Fehleranzahl pro Branch vor. In `evaluation/data_safe/lint_results.json` weisen 4 von 18 Branches 0 ESLint-Fehler aus, während 14 von 18 Branches 1 ESLint-Fehler ausweisen. Eine Aufschlüsselung nach ESLint-Regelkategorien (z. B. *no-unused-vars*, *no-explicit-any*) ist in den Artefakten nicht enthalten und wurde daher nicht ausgewertet.

Als Surrogat für Typisierungsstrenge und potenziellen “Escape” aus dem Typsystem wird die Nutzung von `any` ausgewiesen (`anyUsage` in `evaluation/code_quality_metrics.json`). Über alle Branches liegt `anyUsage` zwischen 2 und 42 (Median: 8.5). Zusätzlich wird die Verwendung von `console` als Indikator für Debug-Ausgaben berichtet (`consoleUsage`). Diese liegt zwischen 4 und 19 (Median: 6). Spezifische Vorkommen von `TODO/FIXME` oder “temporären” Debug-Endpunkten werden in den vorliegenden Metriken nicht erfasst.

6.4.2 Testabdeckung und Teststabilität

Die Testdaten liegen in zwei Formen vor: (a) eine Aggregation von Pass/Fail-Zählwerten je Branch in `evaluation/deep_code_analysis.json` sowie (b) Jest-JSON-Zusammenfassungen je Branch in `evaluation/data_safe/test_results.json`. In `evaluation/deep_code_analysis.json` reichen die pro Branch gezählten `test_pass`-Werte von 0 bis 14 und die `test_fail`-Werte von 9 bis 37. 7 von 18 Branches weisen `test_pass=0` aus. 11 von 18 Branches weisen mindestens einen bestandenen Test aus. Die Jest-JSON-Ausgaben bestätigen dieses Bild in Form von Summen (z. B. `numFailedTestSuites`, `numFailedTests`, `numPassedTests`). Eine normalisierte Kategorisierung der Fehlertypen (z. B. nach “Auth/HTTP-Status”, “Schema”, “Datenpersistenz”) ist in den Artefakten nicht enthalten.

Für die Aufgabenfortschritts-Metrik wird zusätzlich `evaluation/task_pipeline_v3.json` herangezogen. Für die dort ausgewiesene Test-Validierung (`verified`) ergeben sich über alle Branches: T1 ist in 10/18 Branches verifiziert, T2 in 4/18, T4 in 0/18 und T5 in 0/18. Für T5 wird in `evaluation/task_pipeline_v3.json` außerdem `implemented=0/18` ausgewiesen. Bei T3 sind in `evaluation/task_pipeline_v3.json` `verified=13/18` und `implemented=7/18` vermerkt. In den Auswertungsartefakten ist `implemented` dabei als heuristisches, statisches Signal (*Implemented_{static}*) zu verstehen, während `verified` testgebundenes Verhalten abbildet. Entsprechend kann `verified` empirisch größer als `implemented` ausfallen. Für die Trichterdarstellung wird diese Differenz über die in Abschnitt 6.2 definierte Evidenzstufe *Implemented** (*Attempted* $\wedge$ (*Implemented_{static}* $\vee$ *Verified*)) konsistent zusammengeführt.

6.4.3 Architektur- und Strukturindikatoren

Als strukturbezogene Indikatoren werden in `evaluation/deep_code_analysis.json` für einzelne Aufgaben Implementationsklassen sowie Persistenzvarianten ausgewiesen. Für T1 (Persistenz) werden unterschiedliche Backends beobachtet: 14/18 Branches sind als *sqlite* klassifiziert, jeweils 1/18 als *mongo*, *postgres*, *json-file* und *memory/other*. Für T2 (Auth) werden 17/18 Branches als *implemented* und 1/18 als *none* klassifiziert. Für T3 (Kategorien) sind 7/18 Branches als *implemented* und 11/18 als *none* klassifiziert. Für T4 (Attachments) sind 8/18 als *implemented* und 10/18 als *none* klassifiziert. Für T5 (Kalendar) ist in `evaluation/deep_code_analysis.json` über alle 18 Branches hinweg *none* ausgewiesen, konsistent zu `evaluation/task_pipeline_v3.json` (`implemented=0/18`, `verified=0/18`).

Die Codeumfangs-Metrik `tsFileCount` (`evaluation/code_quality_metrics.json`) liegt über alle Branches zwischen 43 und 80 TypeScript-Dateien (Median: 54) und liefert damit einen groben, rein quantitativen Indikator für Codebasis-Variation.

6.4.4 Zusammenhänge mit Task-Fortschritt

Für eine rein deskriptive Gegenüberstellung wird die Anzahl als `implemented` markierter Tasks pro Branch aus `evaluation/task_pipeline_v3.json` genutzt. Die Branches verteilen sich dabei auf 1 bis 4 als `implemented` markierte Tasks (T5 ist in allen Fällen nicht implementiert). In den gruppierten Mittelwerten ko-auftreten höhere Werte von `anyUsage` und `consoleUsage` mit höherer `implemented`-Taskanzahl (z.B. `anyUsage` im Mittel 8.8 bei 2 implementierten Tasks vs. 12.4 bei 3 vs. 16.7 bei 4). Analog zeigt die Gegenüberstellung nach ESLint-Fehleranzahl (0 vs. 1 Fehler laut `evaluation/data_safe/lint_results.json`) Unterschiede in den Medianen von `consoleUsage` (4.5 vs. 7.5) und `anyUsage` (7.5 vs. 9). Diese Darstellung ist als kleine Fallzahl bei 0 Fehlern (4 Branches) zu lesen.

Darüber hinaus enthalten die vorliegenden Artefakte keine Kennzahlen zu Testabdeckung in Prozent, keine Komplexitätsmaße (z.B. zyklomatische Komplexität), keine TypeScript-Compilerfehlerstatistiken, keine Performance-/Bundle-Metriken und keine konsolidierten Frontend-Lighthouse-Ergebnisse. Aus diesem Grund werden hier keine weitergehenden, metrisch gestützten Aussagen über Abdeckung, Komplexität oder Laufzeitverhalten getroffen.

6.4.5 Kurzfazit: Beobachtete Qualitätsmerkmale

- ESLint: 4/18 Branches ohne ESLint-Fehler, 14/18 mit genau einem Fehler (keine Regelkategorien verfügbar).
- Typisierung/Debug-Surrogate: `anyUsage` variiert stark (2–42), `consoleUsage` liegt zwischen 4 und 19.
- Tests: Testausgänge sind heterogen (`test_pass` 0–14 und `test_fail` 9–37). 7/18 Branches sind ohne bestandene Tests.
- Aufgabenvalidierung (Pipeline v3): T1 ist in 10/18 Branches verifiziert, T2 in 4/18. T4 und T5 sind in 0/18 verifiziert.
- Architekturindikatoren: Persistenz- und Feature-Varianten sind sichtbar (mehrere Persistenztypen, T2 überwiegend implementiert, T5 in allen Branches nicht implementiert).

6.5 Chat-Interaktionsmuster

Dieses Unterkapitel beschreibt beobachtbare Muster der Interaktion zwischen den Teilnehmenden (Teilnehmer A–R) und dem im Experiment eingesetzten Tool *GitHub*

Copilot mit dem zugrundeliegenden LLM *Claude Sonnet 4.5*. Als Datenbasis dienen ausschließlich die 18 Chat-Exports in `thesis/chats/` (`.docx`) sowie der Task-Status aus `evaluation/task_pipeline_v3.json`. Ein separates, maschinell generiertes Chat-Metrik-Artefakt liegt nicht vor. Alle berichteten Metriken wurden direkt aus den Chat-Exports extrahiert. Die Zuordnung Chat $\leftrightarrow$ Branch erfolgt über die im Evaluationskapitel beschriebene Normalisierung der Chat-Dateinamen und deren Mapping auf Branches. Alle Aussagen werden als Zählwerte, Spannweiten oder Verteilungskennwerte berichtet. Eine qualitative Inhaltsbewertung der Antworten erfolgt in diesem Unterkapitel nicht.

6.5.1 Umfang und Intensität der Interaktion

Messdefinition. Für alle 18 Chats wurde die Wortanzahl als whitespace-tokenisierte Wortzählung auf dem aus `.docx` extrahierten Text bestimmt. Wortanzahl, Zyklenzahl und Anzahl der Nutzerbeiträge sind in diesem Abschnitt als Intensitätsindikatoren operationalisiert und werden nicht als Leistungs- oder Effizienzmaße interpretiert.

Ergebnis. Die Wortanzahl pro Chat liegt zwischen 4046 und 26833 Wörtern (Median: 7137.5, Quartile: Q1=5330.5, Q3=8862.25).

Messdefinition. Die Anzahl der Prompt-Antwort-Zyklen wurde als Sequenzpaar aus einem Nutzerbeitrag (in den Exports typischerweise markiert durch `You said:` oder einen Namenspräfix `X:`) und der unmittelbar folgenden Modellantwort (in den Exports u. a. markiert durch `GitHub Copilot:` oder `ChatGPT said:`) gezählt. Die genannten Sprecherkennzeichnungen werden ausschließlich zur Segmentierung in Nutzerbeiträge und Modellantworten verwendet.

Ergebnis. Über alle Chats liegt die Anzahl der Prompt-Antwort-Zyklen zwischen 12 und 45 (Median: 16.5, Quartile: Q1=14.25, Q3=19.75).

Messdefinition. Als zusätzlicher Intensitätsindikator wurde die Anzahl der erkannten Nutzerbeiträge pro Chat bestimmt.

Ergebnis. Diese liegt zwischen 53 und 777 Nutzerbeiträgen (Median: 144). Damit werden in den Chat-Exports sowohl Fälle mit wenigen Dutzend Nutzerbeiträgen als auch Fälle mit mehreren hundert Nutzerbeiträgen beobachtet.

6.5.2 Struktur der Prompts

Messdefinition. Für die formale Beschreibung der Prompt-Struktur wurden alle 3390 erkannten Nutzerbeiträge aggregiert ausgewertet. Die folgenden Merkmale wurden regelbasiert bestimmt. Die Kategorien sind nicht disjunkt, ein Nutzerbeitrag kann mehrere Merkmale gleichzeitig aufweisen.

Ergebnis. Ein-Satz-Prompts (operationalisiert als Nutzerbeiträge mit höchstens einem Satzendzeichen `.?!)` treten in 1901 von 3390 Fällen auf (56.1%). Strukturierte Mehrpunkt-Prompts (operationalisiert über Aufzählungs-/Schrittmuster, z. B. `•` oder nummerierte Listen sowie Schlüsselwörter wie `To do/Schritt`) treten in 471 von 3390 Fällen auf (13.9%).

Kontextangaben (operationalisiert über Kontextmarker wie `repo`, `backend`, `frontend`, `docker` oder `workspace`) treten in 1555 von 3390 Fällen auf (45.9%). Explizite Anforde-

rungen/Constraints (operationalisiert über modale/negierende Marker wie **muss/must/do not/ausschließlich**) treten in 307 von 3390 Fällen auf (9.1%). Schrittanweisungen (operationalisiert über Marker wie **Schritt/first/zuerst** oder nummerierte Sequenzen) treten in 382 von 3390 Fällen auf (11.3%).

Nachforderungen bzw. erneute Anweisungen (operationalisiert über Wiederholungs-/Umformulierungsmarker wie **nochmal/anders/rewrite**) treten in 67 von 3390 Fällen auf (2.0%).

6.5.3 Debugging-Interaktionen und Iterationsmuster

Messdefinition. Debugging-bezogene Nutzerbeiträge wurden über Vorkommen typischer Fehlermarker (z. B. **error, exception, failed, TypeError, TS...**, **eslint**) erfasst. Die folgenden Iterations- und Wiederholungsmetriken beschreiben ausschließlich Textmuster in den Nutzerbeiträgen. Daraus werden keine Aussagen über Emotionen, Kompetenz oder Motivation abgeleitet.

Ergebnis. Insgesamt enthalten 296 von 3390 Nutzerbeiträgen solche Fehlermarker (8.7%).

Messdefinition. Als *Iteration* wurde eine Folge von Nutzerbeiträgen operationalisiert, die jeweils Fehlermarker enthalten (aufeinanderfolgende Korrekturversuche im selben Chatabschnitt).

Ergebnis. Über alle Chats hinweg wurden 254 solche Iterationssequenzen gezählt. Die maximale Länge aufeinanderfolgender Fehlermarker-Nutzerbeiträge in einem einzelnen Chat liegt zwischen 1 und 9 (Median: 2, Maximum: 9).

Messdefinition. Als einfache Form von *Wiederholung* wurde zusätzlich erfasst, ob zwei aufeinanderfolgende Fehlermarker-Nutzerbeiträge eine identische normalisierte Fehlersignatur (erste erkannte Error-/Exception-Zeile) enthalten.

Ergebnis. Über alle Chats hinweg wurden 23 unmittelbare Wiederholungen identischer Fehlersignaturen gezählt.

Messdefinition. Als Abbruch-/Wechselindikator wurden Task-Sprünge in den Nutzerbeiträgen über Task-Mentions (**Task 1-8, Aufgabe 1-8, T1-T8**) erfasst. Task-Wechsel werden in diesem Unterkapitel als Strukturmerkmal der Interaktion berichtet und nicht bewertet.

Ergebnis. 148 von 3390 Nutzerbeiträgen enthalten mindestens eine Task-Mention (4.4%). In 8 von 18 Chats wird mindestens ein Wechsel der zuerst genannten Task-Nummer beobachtet. Über alle Chats hinweg wurden 74 solche Task-Wechsel gezählt.

6.5.4 Modelltypische Antwortmuster (Claude Sonnet 4.5)

Messdefinition. Für die Modellantworten wurden 335 erkannte Antworten aggregiert ausgewertet. Als Modellantworten werden Textsegmente erfasst, die in den Exports als Antwort des KI-Assistenzsystems markiert sind (z. B. durch **GitHub Copilot:** oder **ChatGPT said:**). Diese Kennzeichnungen werden ausschließlich zur Segmentierung verwendet.

Ergebnis. Codeblöcke in der Modellantwort (operationalisiert über das Vorkommen von Markdown-Fences "``") treten in 122 von 335 Modellantworten auf (36.4%). Rückfragen bzw. Frageformen (operationalisiert über ein abschließendes Fragezeichen oder Formulierungen wie *Would you like/Can you*) treten in 21 von 335 Modellantworten auf (6.3%).

Tooling-/Aktionsformulierungen (operationalisiert über Marker wie *Ran terminal command* oder *Replace String in File*) treten in 41 von 335 Modellantworten auf (12.2%). Architektur-/Strukturbegriffe (operationalisiert über Schlüsselwörter wie *controller*, *service*, *repository*, *routes*, *docker*, *sqlite*) treten in 152 von 335 Modellantworten auf (45.4%).

6.5.5 Beziehung zwischen Chat-Mustern und Task-Fortschritt (deskriptiv)

Messdefinition. Für eine rein deskriptive Gegenüberstellung wird der Task-Fortschritt je Branch aus *evaluation/task_pipeline_v3.json* neben die Interaktionsmetriken gestellt. Dabei wird *implemented* als heuristisches, statisches Artefakt-Signal (*Implemented_{static}*) verstanden; *verified* bildet testgebundenes Verhalten ab. Die folgenden Aussagen beschreiben Ko-Vorkommen innerhalb derselben Branches. Es werden keine Wirk- oder Kausalzusammenhänge abgeleitet.

Ergebnis. Über die 18 Branches hinweg liegen die *implemented*-(*Implemented_{static}*)-Taskanzahlen zwischen 1 und 4 (T5 ist in *evaluation/task_pipeline_v3.json* in allen Branches nicht implementiert).

In der Gruppe mit 2 *implemented*-(*Implemented_{static}*)-Tasks (n=6) liegt die Wortanzahl zwischen 4046 und 9190 (Median: 6449) und die Zyklenzahl zwischen 12 und 20 (Median: 14.5). Die Anzahl verifizierter Tasks liegt zwischen 0 und 2 (Median: 2). In der Gruppe mit 3 *implemented*-(*Implemented_{static}*)-Tasks (n=8) liegt die Wortanzahl zwischen 4863 und 26833 (Median: 8865) und die Zyklenzahl zwischen 16 und 45 (Median: 18). Die Anzahl verifizierter Tasks liegt zwischen 0 und 2 (Median: 2). In der Gruppe mit 4 *implemented*-(*Implemented_{static}*)-Tasks (n=3) liegt die Wortanzahl zwischen 4764 und 6663 (Median: 4919) und die Zyklenzahl zwischen 14 und 21 (Median: 14). Die Anzahl verifizierter Tasks liegt zwischen 0 und 2 (Median: 2). Die Kombination aus hoher Wortanzahl (bis 26833) und hoher Zyklenzahl (bis 45) tritt innerhalb derselben Gruppe (3 *implemented*-(*Implemented_{static}*)-Tasks) auf.

Dieses Unterkapitel beschreibt ausschließlich beobachtbare Interaktionsmuster und deren Messwerte. Es enthält keine Effizienz- oder Leistungsbewertung, keine Kompetenzzuschreibung, keine psychologischen Deutungen und keine Modellbewertung. Die Interpretation dieser Muster sowie deren Einordnung erfolgt ausschließlich in Kapitel 7.

6.6 Triangulation der Datenquellen

Dieses Unterkapitel stellt die in Abschnitt 6.2 bis 6.5 berichteten Ergebnisse sowie Survey-Merkmale nebeneinander. Ziel ist eine rein deskriptive Triangulation auf Ebene der 18

Branches (Teilnehmer A–R): Pipeline-Fortschritt (*attempted/implemented/verified*), beobachtete Strukturindikatoren der Implementationen, technische Metriken, Interaktionsmetriken aus den Chat-Exports sowie Survey-Angaben. Alle Aussagen werden als Zählwerte, Spannweiten oder Verteilungskennwerte berichtet. Daraus werden keine Wirk- oder Kausalzusammenhänge abgeleitet. Die folgenden Abschnitte ordnen diese Datenquellen paarweise und gruppenweise nebeneinander, um Muster und Spannweiten sichtbar zu machen.

6.6.1 Datenbasis und Aggregationslogik

Messdefinition. Die Triangulation erfolgt pro Branch als gemeinsame Analyseeinheit. Der Pipeline-Fortschritt wird aus `evaluation/task_pipeline_v3.json` als Anzahl der pro Branch als *implemented* bzw. *verified* markierten Tasks (T1–T5) abgeleitet. Dabei wird *implemented* als heuristisches, statisches Artefakt-Signal (*Implemented_{static}*) verstanden; die in Abschnitt 6.2 eingeführte Trichter-Evidenzstufe *Implemented** wird in diesem Unterkapitel nicht als Gruppierungsvariable verwendet. Als ergänzende Strukturindikatoren (z. B. Persistenzvariante) und Test-Summen (`test_pass/test_fail`) wird `evaluation/deep_code_analysis.json` genutzt. Technische Metriken wie `tsFileCount`, `anyUsage` und `consoleUsage` stammen aus `evaluation/code_quality_metrics.json`. Survey-Merkmale (Kompetenzgruppe und Hürdenkategorie) werden aus `thesis/surveys/Probanden-Einstufungsformular(Responses).xlsx` über das Mapping `evaluation/survey_mapping.json` übernommen. Chat-bezogene Metriken werden in diesem Unterkapitel nicht neu berechnet, sondern als die in Abschnitt 6.5 berichteten Werte (Extraktion aus `thesis/chats/`) übernommen.

Ergebnis. Die Branches verteilen sich auf 1 bis 4 als *implemented* markierte Tasks (Verteilung: 1 → 1 Branch, 2 → 6 Branches, 3 → 8 Branches, 4 → 3 Branches). Für *verified* ergeben sich 0 bis 2 verifizierte Tasks je Branch (Verteilung: 0 → 4 Branches, 1 → 1 Branch, 2 → 13 Branches). T5 ist in `evaluation/task_pipeline_v3.json` in allen Branches nicht implementiert und nicht verifiziert.

6.6.2 Pipeline-Fortschritt und Implementationsstrategien (Strukturindikatoren)

Messdefinition. Als Stellvertreter für ausgewählte Implementationsstrategien werden Strukturindikatoren aus `evaluation/deep_code_analysis.json` herangezogen. Für T1 betrifft dies die klassifizierte Persistenzvariante (z. B. *sqlite*, *mongo*). Für T2–T4 werden Implementationsklassen (*implemented/none*) berichtet.

Ergebnis. Über alle Branches hinweg dominiert bei T1 *sqlite* (14/18). Daneben treten *mongo* (1/18), *postgres* (1/18), *json-file* (1/18) und *memory/other* (1/18) auf. Bei Gruppierung nach *implemented*-Taskanzahl zeigt sich folgende Verteilung der Persistenzvarianten: bei 2 implementierten Tasks (n=6) *sqlite* in 5/6 Fällen und *postgres* in 1/6, bei 3 implementierten Tasks (n=8) *sqlite* in 7/8 und *mongo* in 1/8, bei 4 implementierten Tasks (n=3) *sqlite* in 2/3 und *json-file* in 1/3. Der einzelne Branch mit 1 implementiertem Task ist *memory/other*.

Für T2 wird in `evaluation/deep_code_analysis.json` in 17/18 Branches `implemented` ausgewiesen. Konsistent dazu liegen in `evaluation/task_pipeline_v3.json` `implemented` für T2 in 17/18 Branches vor. Für T3 sind 7/18 Branches in `evaluation/task_pipeline_v3.json` als `implemented` (*Implemented_{static}*) markiert. Gleichzeitig ist T3 in 13/18 Branches `verified`. Für T4 sind 8/18 Branches als `implemented` markiert. `verified` ist bei T4 in allen Branches 0/18.

6.6.3 Pipeline-Fortschritt und technische Metriken

Messdefinition. Für eine deskriptive Gegenüberstellung werden pro Branch technische Metriken aus `evaluation/code_quality_metrics.json` (`tsFileCount`, `anyUsage`, `consoleUsage`) sowie Test-Summen aus `evaluation/deep_code_analysis.json` (`test_pass`, `test_fail`) neben die Anzahl `implemented` Tasks gestellt.

Ergebnis. Über alle 18 Branches liegt `tsFileCount` zwischen 43 und 80 Dateien. `anyUsage` liegt zwischen 2 und 42 und `consoleUsage` zwischen 4 und 19. Gruppiert nach `implemented`-Taskanzahl ergeben sich folgende Spannweiten und Mediane:

- 1 implementierter Task (n=1): `anyUsage`=6, `consoleUsage`=6, `tsFileCount`=43, `test_pass`=14, `test_fail`=12.
- 2 implementierte Tasks (n=6): `anyUsage` 2–30 (Median: 4), `consoleUsage` 4–17 (Median: 4.5), `tsFileCount` 43–63 (Median: 56.5), `test_pass` 0–14 (Median: 12) und `test_fail` 9–37 (Median: 11).
- 3 implementierte Tasks (n=8): `anyUsage` 3–42 (Median: 8.5), `consoleUsage` 4–19 (Median: 10), `tsFileCount` 49–80 (Median: 54), `test_pass` 0–12 (Median: 0) und `test_fail` 12–28 (Median: 15.5).
- 4 implementierte Tasks (n=3): `anyUsage` 15–19 (Median: 16), `consoleUsage` 4–9 (Median: 6), `tsFileCount` 47–74 (Median: 57), `test_pass` 0–13 (Median: 12) und `test_fail` 10–13 (Median: 12).

Diese Gegenüberstellung beschreibt ausschließlich gemeinsame Verteilungen innerhalb derselben Branches.

6.6.4 Pipeline-Fortschritt und Chat-Interaktionsmetriken

Messdefinition. Als Chat-Metriken werden die in Abschnitt 6.5 definierten und berichteten Intensitäts- und Interaktionsindikatoren (Wortanzahl, Prompt-Antwort-Zyklen, Nutzerbeiträge, Fehlermarker-Iterationen) herangezogen. Für die Triangulation werden diese Werte neben die `implemented`- und `verified`-Taskanzahl aus `evaluation/task_pipeline_v3.json` gestellt.

Ergebnis. In Abschnitt 6.5 wird bereits eine gruppierte Gegenüberstellung nach `implemented`-Taskanzahl berichtet. Dort liegen (a) in der Gruppe mit 2 implementierten Tasks (n=6) Wortanzahl und Zyklenzahl zwischen 4046–9190 Wörtern (Median: 6449) bzw. 12–20 Zyklen (Median: 14.5), (b) in der Gruppe mit 3 implementierten Tasks (n=8)

zwischen 4863–26833 Wörtern (Median: 8865) bzw. 16–45 Zyklen (Median: 18) und (c) in der Gruppe mit 4 implementierten Tasks (n=3) zwischen 4764–6663 Wörtern (Median: 4919) bzw. 14–21 Zyklen (Median: 14). Über alle Gruppen hinweg liegt die verifizierte Taskanzahl in `evaluation/task_pipeline_v3.json` je Branch zwischen 0 und 2.

6.6.5 Pipeline-Fortschritt und Survey-Merkmale

Messdefinition. Aus dem Survey werden (a) die Kompetenzgruppe (No-Code, Low-Code, Mid-Code, High-Code) und (b) eine primäre Hürdenkategorie (Survey-Feld D) als kategoriale Merkmale herangezogen. Beide Merkmale werden ausschließlich als Gruppierungs- bzw. Häufigkeitsvariablen genutzt.

Ergebnis. Die Kompetenzgruppen verteilen sich auf No-Code (5/18), Low-Code (4/18), Mid-Code (5/18) und High-Code (4/18). Tabelle 6.3 stellt diese Gruppen der `implemented`-Taskanzahl gegenüber.

Kompetenzgruppe	1 impl.	2 impl.	3 impl.	4 impl.
No-Code (n=5)	0	2	2	1
Low-Code (n=4)	1	1	2	0
Mid-Code (n=5)	0	1	2	2
High-Code (n=4)	0	2	2	0

Tabelle 6.3: Kompetenzgruppe (Survey) vs. Anzahl `implemented` (*Implemented_{static}*) Tasks pro Branch (`evaluation/task_pipeline_v3.json`, $N = 18$).

Als primäre Hürdenkategorien (Survey-Feld D) treten am häufigsten Zeitdruck (5/18) und Debugging von Fehlern (5/18) auf, gefolgt von Aufgabenverständnis (3/18). In der Gruppe mit 2 implementierten Tasks (n=6) ist Zeitdruck 2/6-mal und Debugging 1/6-mal als primäre Hürde genannt. In der Gruppe mit 3 implementierten Tasks (n=8) ist Debugging 3/8-mal und Zeitdruck 2/8-mal als primäre Hürde genannt. In der Gruppe mit 4 implementierten Tasks (n=3) treten Zeitdruck, Debugging und Aufgabenverständnis jeweils in 1/3 Fällen als primäre Hürde auf.

6.6.6 Abgrenzung zu Kapitel 7

Dieses Unterkapitel stellt ausschließlich Ko-Vorkommen und Verteilungen über mehrere Datenquellen hinweg dar. Es enthält keine Leistungs- oder Effizienzbewertung, keine Kompetenzzuschreibung und keine kausale Interpretation. Die Einordnung dieser Muster sowie die Diskussion möglicher Erklärungen erfolgt ausschließlich in Kapitel 7.

6.7 Zusammenfassung der Ergebnisse

Kapitel 6 dokumentiert die Ergebnisse der Evaluation und stellt beobachtete Ausprägungen und Verteilungen dar, ohne diese zu bewerten oder zu erklären. Die Ergebnisdarstellung basiert auf mehreren Datenquellen (Pipeline-Status `attempted/implemented/verified`,

Code- und Test-Artefakte, Chat-Exports sowie Survey-Merkmale) und bleibt durchgängig deskriptiv. Im Folgenden werden die zentralen Befundlinien systematisch zusammengeführt.

Pipeline-Ergebnisse entlang der Tasks (T1–T5). Über die Tasks hinweg zeigt sich ein gemeinsamer Trichter-Effekt: Attempted-Markierungen sind in der Breite vorhanden, während die Anteile der als Implemented* (Trichter-Evidenzstufe) bzw. Verified ausgewiesenen Ergebnisse mit zunehmendem Integrations- und Abhängigkeitsgrad abnehmen. Dabei ist die Ausprägung dieses Trichters nicht in allen Branches gleichförmig, sondern zeigt Spannweiten zwischen sehr kurzem Task-Fortschritt und einem deutlich weiter reichenden Umsetzungsstand. In der Gegenüberstellung der Aufgaben treten dabei zwei Bündel hervor:

- **Backend-nahe Tasks (T1/T2).** T1 ist in den Branches in unterschiedlichen Persistenzvarianten umgesetzt, wobei eine Variante dominiert und mehrere Alternativen vereinzelt auftreten. T2 wird in der Mehrzahl der Branches als Implemented* ausgewiesen und ist damit ein häufig realisiertes Backend-Element.
- **Integrations- und Frontend-nahe Tasks (T3–T5).** Bei T3 und T4 nehmen die Implemented_{static}-Ausweise ab. Zugleich werden bei einzelnen Tasks Diskrepanzen zwischen Implemented_{static} (heuristische Artefakt-Markierung) und Verified sichtbar (ohne dass daraus in Kapitel 6 Schlussfolgerungen abgeleitet werden). T5 bleibt in den Artefakten durchgängig ohne Implemented_{static}- und ohne Verified-Ausweise.

Über alle Tasks hinweg gilt: Verified, Implemented_{static} und Attempted sind empirisch unterscheidbare Statuskategorien und fallen nicht notwendigerweise zusammen. Für die Trichterdarstellung wird Implemented* ($\text{Attempted} \wedge (\text{Implemented}_{\text{static}} \vee \text{Verified})$) als konsistente Evidenzstufe genutzt. Kapitel 6 beschreibt diese Statusausweise als Ergebnisvariablen der Pipeline und nutzt sie als gemeinsame Referenz, um Code-, Test- und Chat-Artefakte pro Branch in Beziehung zu setzen.

Technische Qualität (aggregiert, ohne Kausalannahmen). Die in Kapitel 6 berichteten Qualitäts- und Technikindikatoren zeigen insgesamt eine heterogene Ausprägung über die Branches hinweg. Dazu gehören (a) Unterschiede im Linting-Status und in der Häufigkeit typischer TypeScript-Surrogate (z. B. Any-Nutzung), (b) Spannweiten bei strukturbezogenen Metriken (z. B. Anzahl TypeScript-Dateien) sowie (c) variierende Testausgänge bis hin zu Konstellationen mit Fehlanteilen. Diese Befunde werden in Kapitel 6 als Ko-Existenz von Pipeline-Fortschritt und technischen Merkmalen beschrieben. Es wird keine Korrelation oder Wirkbeziehung behauptet. Zusätzlich wird sichtbar, dass sich technische Ausprägungen nicht auf einen einheitlichen "Projektzustand" reduzieren lassen: Branches mit ähnlichem Pipeline-Fortschritt können unterschiedliche Muster in Linting-, Typisierungs- und Testindikatoren aufweisen. Zugleich werden die Messgrenzen betont: Die verwendeten Indikatoren ersetzen keine Coverage-, Komplexitäts- oder Performance-Metriken und erlauben entsprechend keine Aussagen zu nicht gemessenen Qualitätsdimensionen.

Chat-Interaktionsmuster (kompakt). Die Chat-Analysen zeigen große Spannweiten bei der Interaktionsintensität, insbesondere bei Wortanzahl und Prompt-Antwort-Zyklen. Über die Branches hinweg koexistieren kurze, wenig strukturierte Prompts mit detaillierten Mehrpunkt-Prompts. Beide Formen treten im selben Datensatz nebeneinander auf. Als wiederkehrendes Strukturmerkmal sind Debugging-Iterationen sichtbar (z. B. wiederholte Fehlermarker und erneute Anpassungsversuche), ohne dass daraus eine qualitative Einordnung ("gut/schlecht") abgeleitet wird. Neben Intensitätsmaßen werden in Kapitel 6 auch Muster in der Interaktionsstruktur beschrieben, etwa wiederkehrende Task-Referenzen, Wechsel zwischen Aufgaben sowie Tooling-/Aktionsformulierungen im Verlauf der Chats.

Triangulation auf Meta-Ebene. Die Triangulation macht sichtbar, dass unterschiedliche Datenquellen teils kongruente, teils divergente Signale liefern: Pipeline-Status, technische Indikatoren, Testausgänge, Chat-Merkmale und Survey-Merkmale können für dieselben Branches zusammenpassen oder auseinanderlaufen. Insbesondere verdeutlicht die Gegenüberstellung, dass $\text{Verified} \neq \text{Implemented}_{\text{static}} \neq \text{Attempted}$ und dass Spannungsfelder zwischen Statusausweisen, technischen Beobachtungen und Interaktionsmustern empirisch auftreten. Auch Survey-Merkmale werden als kategoriale Kontextvariablen einbezogen (Selbsteinstufungsgruppen und benannte Hürdenkategorien), ohne dass daraus individuelle Zuschreibungen oder Bewertungen abgeleitet werden. In der Gesamtschau entstehen damit nebeneinander stehende Ergebnislinien, die Gemeinsamkeiten und Unterschiede zwischen Branches sichtbar machen, ohne diese zu erklären. Kapitel 6 stellt diese Spannungsfelder dar, löst sie jedoch nicht auf.

Abgrenzung zu Kapitel 7. Kapitel 6 beschreibt, *was* beobachtet wurde. Kapitel 7 diskutiert, *wie* diese Befunde einzuordnen sind und *warum* bestimmte Muster auftreten könnten.

7 Diskussion

7.1 Ziel und Abgrenzung der Diskussion

Kapitel 6 hat die Ergebnisse der Studie ausschließlich deskriptiv berichtet und die beobachteten Muster entlang mehrerer Datenquellen nebeneinandergestellt. Kapitel 7 interpretiert diese Befunde im Licht des in Kapitel 2 eingeführten theoretischen Rahmens. Ziel ist es, die Forschungsfragen argumentativ zu beantworten und die Hypothesen zu bewerten. Damit adressiert die Diskussion die in Abschnitt 2.6 formulierte Forschungslücke, indem Befunde aus einem kontrollierten Brownfield-Setting entlang der Pipeline-Status und im Kompetenzkontext eingeordnet werden, ohne daraus eine Generalisierbarkeit über andere Settings oder KI-Assistenzsysteme abzuleiten. Dabei werden keine neuen Daten eingeführt. Es werden keine zusätzlichen Metriken berechnet. Es werden keine Kausalzusammenhänge behauptet, die durch das Studiendesign oder die erhobenen Artefakte nicht abgesichert sind.

Methodische Selbstbegrenzung (Kausalität, Gruppenvergleiche, Kompetenzgruppen). Die Studie ist nicht als kausales Design angelegt, sondern als kontrollierte, artefaktbasierte Beobachtungsstudie: Es gibt keine Randomisierung, keine experimentelle Manipulation und keine prozessnahen Messungen, die eine Identifikation von Ursache-Wirkung erlauben würden. Zudem sind zentrale Einflussgrößen (z.B. Technologie-Fit, Test-/Debugging-Routinen, Prompting-Erfahrung, Unterbrechungen) nur teilweise messbar und können daher nicht hinreichend kontrolliert werden. Aus diesem Grund werden Befunde konsistent als deskriptive Muster und theoriegeleitete Einordnungen berichtet. Auf inferenzstatistische Gruppenvergleiche (Signifikanztests, Effektstärken) wird verzichtet, da die Fallzahlen pro Kompetenzgruppe klein sind, die Messannahmen für stabile Schätzungen nicht belastbar geprüft werden können und multiple Konfundierungen das Risiko scheinbarer Gruppenunterschiede erhöhen. Die Kompetenzgruppen werden deshalb nicht als Effektvariablen im Sinne gruppenstabiler Leistungs- oder Qualitätsunterschiede modelliert, sondern als Analyseperspektiven zur Strukturierung von Nutzungstypen und Interpretationskontexten. Damit bleibt die Diskussion auf die Frage fokussiert, welche Artefaktspuren und Prozesssignale im Brownfield-Setting beobachtbar sind und wie diese im Kompetenzkontext plausibel eingeordnet werden können.

Geltungsbereich der Ergebnisse. Die in dieser Arbeit diskutierten Befunde beziehen sich ausschließlich auf *interaktive* KI-Assistenzsysteme im Vibe-Coding-Kontext, die in einer Brownfield-Sandbox auf ein bestehendes Codeartefakt angewendet werden. Sie gelten für eine zeitlich begrenzte Aufgabenbearbeitung unter den in Kapitel 6 beschriebenen

Rahmenbedingungen. Nicht Gegenstand der Aussagen sind Greenfield-Projekte oder langfristige Produktentwicklung. Ebenfalls nicht abgedeckt sind autonome Agenten mit eigener Aufgabenplanung und eigenständiger Ziel- und Teilzielbildung.

Eine zentrale Linse der Diskussion sind die vier Kompetenzgruppen No-Code, Low-Code, Mid-Code und High-Code. Diese Gruppen werden als gleichwertige Analyseperspektiven behandelt. Sie sind keine Qualitätsstufen und dienen nicht der Bewertung einzelner Personen. Aussagen beziehen sich daher auf aggregierte Gruppenmuster oder auf einzelne Branches als Analyseeinheiten. Eine Leistungsbewertung oder Rangordnung einzelner Personen erfolgt nicht.

Group comparability & confounders. Die Vergleichbarkeit der Kompetenzgruppen ist in dieser Studie nur in Teilen abgesichert. Auf der einen Seite sind zentrale Rahmenbedingungen für alle Teilnehmenden konstant gehalten (einheitliche IDE „VS Code“, identische Sandbox und Aufgabenstellung, identisches Zeitlimit, sowie dasselbe KI-Assistenzsystem aus GitHub Copilot und Claude Sonnet 4.5). Damit sind Unterschiede in den in Kapitel 6 berichteten Artefaktspuren und Pipeline-Status grundsätzlich *kompatibel mit* kompetenzabhängigen Nutzungstypen. Auf der anderen Seite können alternative Erklärungen nicht ausgeschlossen werden: Unterschiede in Technologie-Fit (TS/Node/Backend), allgemeiner Berufserfahrung, Test-/Debugging-Routinen und individueller Risikotoleranz beeinflussen sowohl Implementationsentscheidungen als auch die Wahrscheinlichkeit, *Verified* unter der gegebenen Testmethodik zu erreichen. Einige dieser Faktoren sind im Pre-Survey als Kontextmerkmale verfügbar (z.B. Erfahrung, Technologie-Fit) und können deskriptiv als Gruppenprofil berichtet werden; andere (z.B. Vorerfahrung mit Copilot/LLMs, Prompting-Strategiequalität, Hardware/Performance, Unterbrechungen) sind in den vorliegenden Artefakten nicht belastbar operationalisiert. Entsprechend sind die Diskussionen in Kapitel 7 als theoriegeleitete Einordnungen zu lesen: Unterschiede *lassen sich interpretieren als* unterschiedliche Integrations- und Validierungsmodi im Kompetenzkontext, ohne daraus kausale Effekte oder gruppenstabile Leistungsrangfolgen abzuleiten.

Zur begrifflichen Trennung wird – wie in Kapitel 1 eingeführt – zwischen Assistenzumgebung (Tool), Modell (LLM) und *KI-Assistenzsystem* als funktionaler Einheit unterschieden.

7.2 Rückbindung an die Hypothesen

Die Hypothesen wurden im Forschungsdesign als Erwartung darüber formuliert, wie Kompetenz und Arbeitsmodus mit dem KI-Assistenzsystem zusammenwirken. Im Folgenden wird jede Hypothese anhand der in Kapitel 6 berichteten Befundlinien diskutiert. Dabei werden die vier Kompetenzgruppen No-Code, Low-Code, Mid-Code und High-Code als Analyseperspektiven verwendet. Nicht jede Hypothese zielt in gleicher Weise auf alle Gruppen.

7.2.1 H1: Mid-Code als Sweet Spot

Befunde, die H1 stützen. Die Ergebnisse aus Kapitel 6 sprechen für ein Muster, das mit einer produktiven Balance zwischen Delegation und Kontrolle bei Mid-Code vereinbar ist. In der Triangulation erscheinen für Mid-Code sowohl substanzielle Umsetzungsstände als auch Hinweise auf systematische Validierung als plausibles Arbeitsmuster. Dieses Bild ist kompatibel mit der in Kapitel 2 beschriebenen Kompetenzlogik, nach der Mid-Code Probleme in Teilprobleme zerlegt und die durch das KI-Assistenzsystem erzeugten prozeduralen Chunks fachlich prüfen kann. Die in Kapitel 6 beobachtete Koexistenz von Implementationsartefakten und Debugging-Iterationen ist damit konsistent.

Befunde, die H1 relativieren. Die Gegenüberstellung Kompetenzgruppe und Umsetzungsstand zeigt kein exklusives Muster, das nur Mid-Code zugeordnet werden kann. Auch in anderen Gruppen treten Branches mit weiter reichenden Umsetzungsständen auf. Damit ist das empirische Signal nicht stark genug, um Mid-Code als eindeutig vorteilhaften Nutzungstyp gegenüber den anderen Analyseperspektiven zu interpretieren. Zudem sind die in Kapitel 6 ausgewiesenen Qualitätsindikatoren nur Surrogate. Sie erlauben keine direkte Aussage darüber, ob Mid-Code tatsächlich systematischer oder effizienter validiert.

Bewertung. H1 wird insgesamt teilweise gestützt. Die Befunde sind kompatibel mit einem Sweet-Spot-Argument. Sie sind jedoch nicht hinreichend spezifisch, um eine klare Stützung gegenüber den anderen Kompetenzgruppen zu begründen. Damit kann die Arbeit H1 als theoriekompatibles Muster im Brownfield-Setting auf Ebene von Pipeline- und Artefaktspuren einordnen; offen bleibt, ob und unter welchen Variationen von Aufgaben, Zeitbudget und prozessnahen Messungen ein solcher Sweet Spot replizierbar und trennscharf nachweisbar ist.

7.2.2 H2: Validierungsparadoxon bei Low-Code

Befunde, die H2 stützen. Kapitel 6 zeigt wiederholt Spannungsfelder zwischen sichtbarer Implementationsaktivität und fehlender testbasierter Bestätigung, insbesondere bei konfigurationssensitiven und stark integrierten Aufgaben. Die Befunde sind vereinbar mit der Interpretation, dass eine rasche Generierung von Artefakten nicht automatisch mit stabiler, testkonformer Funktionalität einhergeht. Dies ist kompatibel mit Kapitel 2, in dem Vibe-Coding als System 1-Aktivität und Automation Bias als Risiko unkritischer Übernahme diskutiert wird. Für Low-Code lässt sich dies theoriegeleitet damit plausibilisieren, dass syntaktische Plausibilität häufig leichter zu beurteilen ist als semantische Korrektheit in einem bestehenden System.

Befunde, die H2 relativieren. Die Artefakte in Kapitel 6 erlauben keine direkte Beobachtung individueller Validierungspraktiken, etwa durch Protokolle von Testläufen oder IDE-Aktionen. Auch werden Chat- und Code-Metriken nicht systematisch nach Kompetenzgruppe aufgeschlüsselt. Damit bleibt die Zuordnung des Validierungsparadoxons zu Low-Code eine theoriegeleitete Interpretation. Es handelt sich nicht um einen unmittelbar gruppenspezifisch gemessenen Effekt.

Bewertung. H2 ist plausibel und wird durch die in Kapitel 6 berichteten Diskrepanzen

zwischen Statuskategorien inhaltlich gestützt. Aufgrund der fehlenden gruppenspezifischen Validierungsdaten bleibt die Hypothese jedoch nicht eindeutig. Damit kann die Arbeit die analytische Relevanz der Trennung von Implementationsartefakten und testgebundener Bestätigung für H2 sichtbar machen; offen bleibt, ob und in welchem Ausmaß das Muster in prozessnahen Daten tatsächlich kompetenzspezifisch durch konkrete Validierungspraktiken getrieben ist.

7.2.3 H3: Skepsis und Qualitätsfokus bei High-Code

Befunde, die H3 stützen. Kapitel 6 liefert Hinweise, die mit der Beobachtung vereinbar sind, dass anspruchsvolle Integrationsaufgaben unter Zeitdruck häufig nicht bis zur testbasierten Stabilisierung gelangen. Für High-Code ist im Rahmen von Kapitel 2 ein Arbeitsmodus plausibel, der Vorschläge stärker hinterfragt und damit mehr kognitive Ressourcen in Kontrolle investiert. Dieses Muster wäre vereinbar mit kalibriertem Vertrauen, bei dem Overtrust vermieden wird. Es wäre ebenfalls vereinbar mit dem in Kapitel 2 diskutierten Befund, dass das Modifizieren von KI-Code kognitiv teuer sein kann.

Befunde, die H3 relativieren. Die in Kapitel 6 berichteten Chat-Muster und Qualitätsindikatoren werden nicht nach Kompetenzgruppen getrennt. Daraus folgt, dass ein erhöhter Qualitätsfokus von High-Code nicht direkt beobachtet, sondern nur aus der Theorie abgeleitet werden kann. Zudem zeigen die Ergebnisse keine eindeutige Dominanz von High-Code beim Umsetzungsstand. Ein höherer Qualitätsanspruch kann unter Zeitlimit sogar zu konservativen Entscheidungen führen, ohne dass dies als geringere Kompetenz zu interpretieren ist.

Bewertung. H3 ist im theoretischen Rahmen gut begründbar. Die empirischen Befunde aus Kapitel 6 sind mit dieser Hypothese vereinbar, stützen sie jedoch nicht eindeutig. Damit kann die Arbeit H3 als vorsichtige, theoriegeleitete Erklärung für mögliche Kontroll- und Integrationsmodi im Zeitlimit diskutieren; offen bleibt, ob sich ein entsprechender Qualitätsfokus in prozessnahen Messungen (z.B. Review-Tiefe, Testlauf-Sequenzen) als distinktes Muster empirisch abgrenzen lässt.

7.3 Kompetenzabhängige Interaktionsmuster

Kapitel 6 zeigt, dass Branches sich nicht nur im Umsetzungsstand unterscheiden, sondern auch in der Art der Interaktion, in der Struktur der Prompts und in der Häufigkeit von Debugging-Sequenzen. Diese Variation lässt sich im Licht von Kapitel 2 als Ausdruck unterschiedlicher Kompetenzlagen interpretieren. Die folgenden Deutungen sind explizit theoriegeleitet und formulieren keinen kausalen Anspruch. Sie beschreiben Nutzungstypen und Analyseperspektiven, nicht Qualitätsstufen oder individuelle Leistungsfähigkeit. Zentral ist dabei nicht das Tool selbst, da alle Teilnehmenden mit derselben Assistenzumgebung (GitHub Copilot) und derselben Modellkonfiguration gearbeitet haben. In dieser Einordnung ist vielmehr relevant, welche kognitiven Ressourcen und welche mentalen Modelle zur Verfügung stehen, um Vorschläge zu prüfen, zu integrieren und bei Bedarf

zu korrigieren.

7.3.1 No-Code

Die Befunde aus Kapitel 6 lassen sich so lesen, dass für No-Code der Zugang zu umsetzungsnahen Artefakten erleichtert werden kann. Gleichzeitig kann Validierung und Integration im bestehenden System häufig anspruchsvoll bleiben. Im Rahmen von Kapitel 2 lässt sich dies als Vibe-Coding-Arbeitsmodus interpretieren, in dem das KI-Assistenzsystem prozedurale Chunks liefert, die fehlende Produktionsregeln überbrücken. Gleichzeitig kann ohne tragfähige Schemata der Intrinsic Load der Prüfung und Integration höher ausfallen. Damit ist es im Sinne von Kapitel 2 plausibel, dass System 2 situativ nicht in dem Maß aktiviert wird, das für semantische Validierung erforderlich wäre, obwohl die Oberfläche plausibel wirkt. Diese Einordnung bleibt theoriegeleitet, da Kapitel 6 keine direkten Prozessmessungen der Validierung auf individueller Ebene enthält.

7.3.2 Low-Code

Kapitel 6 lässt sich so lesen, dass Low-Code sichtbare Implementationsartefakte erzeugt, während testgebundene Bestätigung bei stärker integrierten Aufgaben nicht durchgängig erreicht wird. Im Rahmen von Kapitel 2 lässt sich dies als Spannungsfeld zwischen Vertrauen und Kontrolle interpretieren. Dabei kann das KI-Assistenzsystem als Beschleuniger genutzt werden, während semantische Korrektheit im Systemkontext schwerer zu prüfen bleibt. Wenn Validierungsroutinen nicht etabliert sind, ist in dieser theoretischen Lesart plausibel, dass unkalibriertes Vertrauen häufiger auftritt, ohne dass dies aus den Artefakten als gruppenspezifischer Effekt gemessen werden kann. Die in Kapitel 6 berichteten Strategiemuster, etwa tutorial-nahe Implementationsflüsse und Boilerplate, sind mit einem stärker template-basierten Arbeitsmodus vereinbar. Diese Einordnung bleibt theoriegeleitet, da Kapitel 6 Validierungshandlungen nicht gruppenspezifisch und prozessnah erfasst.

7.3.3 Mid-Code

Kapitel 6 ist mit der Beobachtung kompatibel, dass Mid-Code sowohl Umsetzungsartefakte als auch Iterationen der Nachjustierung und Validierung zeigt. Im Rahmen von Kapitel 2 lässt sich dies als Kompetenzlage interpretieren, die zielorientierte Zerlegung und aktive Kontrolle kombiniert. Damit lässt sich das KI-Assistenzsystem eher als Werkzeug in einem kontrollierten Problemlöseprozess einordnen. Die in Kapitel 6 berichtete Vielfalt an Prompt-Formen ist als funktionale Variation plausibel, in der kurze Prompts Beschleunigung und strukturierte Prompts Spezifikation und Kontextabgleich unterstützen. Debugging-Iterationen sind dabei als normaler Bestandteil iterativer Integrationsarbeit zu lesen, nicht als Effizienzindikator. Diese Einordnung bleibt vorsichtig, da Kapitel 6 keine direkten Messungen von Review- und Validierungstiefe enthält.

7.3.4 High-Code

Kapitel 6 ist kompatibel mit dem Befund, dass unter Zeitlimit bei anspruchsvollen Integrationsaufgaben testgebundene Bestätigung nicht durchgängig erreicht wird. Im Rahmen von Kapitel 2 lässt sich dies für High-Code als Nutzungstyp interpretieren. In dieser Einordnung wird das KI-Assistenzsystem eher für Boilerplate und Exploration genutzt, während zentrale Entscheidungen und Integrationslogik stärker manuell geprüft werden. In dieser Perspektive können intensives Prüfen und konservative Integrationsentscheidungen mit geringerem sichtbarem Fortschritt koexistieren, ohne dass daraus unmittelbar Rückschlüsse auf Qualität oder Kompetenz abzuleiten wären. Diese Einordnung bleibt vorsichtig, da Kapitel 6 keine direkte Messung von Review-Tiefe oder Kontrollhandlungen enthält.

7.4 Interpretation der Pipeline-Muster

Die Pipeline-Kategorien *Attempted*, *Implemented_{static}* und *Verified* strukturieren die Befunde in Kapitel 6 entlang eines Trichters; für die Trichterdarstellung wird dort zusätzlich *Implemented** als abgeleitete Evidenzstufe verwendet. Die folgenden Deutungen binden diese deskriptiven Status-Diskrepanzen explizit an die in Kapitel 2 eingeführten Konzepte (u.a. Cognitive Load, Automation Bias) zurück. Für die Diskussion ist entscheidend, dass diese Kategorien unterschiedliche Aspekte desselben Arbeitsprozesses repräsentieren. In der Notation gilt: $Attempted \neq Implemented_{static} \neq Verified$. *Attempted* beschreibt Intention und Planbildung auf der Chat-Ebene. *Implemented_{static}* beschreibt aufgabenspezifische Artefakte in der Codebasis (White-Box). *Verified* beschreibt testgebundenes, von außen beobachtbares Verhalten (Black-Box). *Verified* ist damit methodengebunden und als Integrationsindikator zu lesen, nicht als Leistungs- oder Qualitätsmaß. Aus Kapitel 6 folgt, dass diese Ebenen nicht zusammenfallen müssen.

7.4.1 Attempted ist nicht gleich Implemented_{static}

Die beobachtete Differenz zwischen *Attempted* und *Implemented_{static}* lässt sich plausibel als Integrationsphänomen interpretieren. Insbesondere Aufgaben, die mehrere Systemteile koppeln, gehen mit zusätzlichem Koordinationsaufwand einher. Im Rahmen von Kapitel 2 lässt sich dies über Cognitive Load erklären. Die Aufgabe reduziert sich nicht auf das Erzeugen von Code, sondern umfasst das Verstehen des bestehenden Kontexts, das Abstimmen von Datenmodellen und Schnittstellen sowie das Beheben von Fehlanpassungen. Dieser Aufwand dürfte den Intrinsic Load erhöhen. Das KI-Assistenzsystem kann Extraneous Load senken, indem es Syntax und Boilerplate generiert. Es kann die semantische Konsistenzarbeit im Systemkontext jedoch nicht ersetzen.

7.4.2 Implemented_{static} ist nicht gleich Verified

Die Diskrepanz zwischen *Implemented_{static}* und *Verified* ist nicht als Qualitätsurteil über einzelne Branches zu lesen. Kapitel 6 betont, dass *Verified* ein Diagnosesignal innerhalb

der eingesetzten Testmethodik ist. Für konfigurationssensitive Aufgaben kann Verified stark von Details des Endpunktvertrags und der Testannahmen abhängen. Ein fehlendes Verified-Signal kann daher bedeuten, dass eine Lösung heterogen, nur teilweise integriert oder nicht exakt testkonform ist. Es kann auch bedeuten, dass die Tests eine bestimmte Implementationsform voraussetzen, die nicht mit allen plausiblen Lösungen kompatibel ist.

7.4.3 Warum Verified gleich null kein eindeutiges Negativsignal ist

Kapitel 6 zeigt für bestimmte Integrationsaufgaben, dass testbasierte Bestätigung ausbleibt. Im Kontext dieser Studie ist das als erwartbares Ergebnis eines strikten Zeitlimits und hoher Schnittstellenkomplexität interpretierbar. Gerade in einer Brownfield-Umgebung dürfte der Aufwand durch bestehende Konventionen, vorhandene Datenmodelle und implizite Vertragsannahmen steigen. Dies kann mit erhöhter kognitiver Belastung und Debugging-Aufwand einhergehen. Unter Automation Bias Perspektive ist zudem plausibel, dass scheinbar fertige Artefakte in einzelnen Fällen zu früh als ausreichend akzeptiert werden. Die Abwesenheit von Verified ist in dieser Arbeit daher kein Synonym für fehlendes Verständnis, sondern kann als Signal für Integrationskosten und Validierungsengpässe gelesen werden.

7.5 Einordnung der Chat-Interaktionsmuster

Die Chat-Ergebnisse aus Kapitel 6 zeigen starke Variation der Interaktionsintensität und wiederkehrende Muster von Debugging-Sequenzen. Kapitel 6 bewertet diese Muster nicht. Für die Diskussion sind drei Punkte zentral.

7.5.1 Warum Chat-Intensität kein Effizienzmaß ist

Eine hohe Interaktionsintensität kann mehrere Bedeutungen haben. Sie kann auf Exploration, auf unklare Anforderungen, auf Fehlanpassungen an den Kontext oder auf konsequente Validierung und Nachjustierung hinweisen. Kapitel 2 betont, dass Vibe-Coding den Schwerpunkt vom Schreiben zum Lesen und Prüfen verschiebt. Damit kann ein längerer Chat-Verlauf auch Ausdruck von Kontrolle und Review sein. Umgekehrt kann ein kurzer Chat-Verlauf sowohl effiziente Beschleunigung als auch unkritische Übernahme bedeuten. Ohne zusätzliche Prozessdaten, etwa Messungen von Testläufen oder IDE-Interaktionen, ist eine Effizienzdeutung nicht abgesichert.

7.5.2 Warum Debugging-Loops normal sind

Kapitel 6 berichtet Debugging-Iterationen als wiederkehrendes Strukturmerkmal. Im Rahmen von Kapitel 2 ist dies erwartbar. Mixed-Initiative Interaction bedeutet, dass Vorschläge, Rückfragen und Korrekturen in Schleifen organisiert sind. Zudem kann die Arbeit in einem bestehenden System die Wahrscheinlichkeit von Konfigurations- und

Schnittstellenfehlern erhöhen. Debugging-Sequenzen sind daher als normaler Bestandteil von Integrationsarbeit zu interpretieren. Sie sind nicht per se ein Indikator für geringe Kompetenz oder geringe Qualität.

7.5.3 Explorative und zielgerichtete Interaktion

Kapitel 2 unterscheidet Interaktionsmodi wie Exploration und Beschleunigung. Kapitel 6 zeigt sowohl kurze, wenig strukturierte Prompts als auch strukturierte, mehrpunktige Prompts. Diese Koexistenz lässt sich als situativ wechselnder Arbeitsmodus interpretieren. Exploration kann genutzt werden, um APIs, Projektkonventionen oder mögliche Lösungswege zu verstehen. Beschleunigung kann genutzt werden, um bekannte Muster schnell zu implementieren. Der Wechsel zwischen diesen Modi ist in einer zeitlich begrenzten Aufgabenbearbeitung plausibel.

7.6 Implikationen für Praxis und Forschung

Aus der Diskussion lassen sich vorsichtige Implikationen für Praxis und Forschung formulieren, die über die konkrete Sandbox hinausreichen. Sie betreffen sowohl die Einführung von KI-Tools in Organisationen als auch die Art, wie deren Nutzen evaluiert wird. Sie knüpfen an die in Kapitel 6 berichteten Befunde an und werden im Licht der in Kapitel 2 eingeführten Konzepte formuliert.

7.6.1 Implikationen für Unternehmen

Die Bereitstellung eines KI-Tools allein adressiert nicht automatisch die in Kapitel 6 sichtbaren Unterschiede in Nutzungstypen. Die Befunde sind kompatibel mit der Annahme, dass Kompetenz den Umgang mit Delegation, Kontrolle und Validierung mitprägt. Als Implikation lässt sich vorsichtig ableiten, dass organisatorische Einbettung und Qualifizierung den beobachteten Prozessunterschieden begegnen können. Dies kann sich etwa in Kompetenzaufbau zu Validierung, Testdenken und Schnittstellenverträgen sowie in expliziten Erwartungen an Review und Integrationsarbeit ausdrücken.

7.6.2 Implikationen für die Evaluation von KI-Coding-Tools

Kapitel 6 zeigt, dass einzelne Signale wie Teststatus, Linting oder Interaktionsintensität isoliert schwer interpretierbar sind. Für Forschung und Praxis ergibt sich daraus, dass Interpretationen belastbarer werden, wenn mehrere Perspektiven gemeinsam betrachtet werden. Insbesondere ist die Unterscheidung zwischen Intention, Implementationsartefakt und testbasierter Bestätigung zentral. Ebenso ist der Kompetenzkontext für die Einordnung der Artefakte relevant, weil aggregierte Durchschnittseffekte Nutzungstypen überdecken können.

7.6.3 Abgrenzung zu Marketing-Narrativen

In der öffentlichen Darstellung werden häufig generische Produktivitätsgewinne hervorgehoben. Die in Kapitel 6 berichteten Muster legen dagegen eine differenzierte Einordnung entlang von Integrationsaufwand, Validierbarkeit und Kompetenzkontext nahe. Im Rahmen von Kapitel 2 können KI-Assistenzsysteme Extraneous Load reduzieren und damit Umsetzungsschritte beschleunigen. Gleichzeitig verschieben sich Anforderungen hin zu Review, semantischer Validierung und testmethodisch gebundener Absicherung. Ohne diese Prozessanteile ist im Sinne von Kapitel 2 unkalibriertes Vertrauen plausibel, ohne dass dies kausal belegt wäre. Damit ist der Nutzen nicht als automatische Eigenschaft eines Tools zu verstehen, sondern als Ergebnis einer arbeitsorganisatorischen Einbettung, die Kompetenzlagen und Integrationskosten berücksichtigt.

7.7 Limitationen und Ausblick

Die Studie ist in ihrer Aussagekraft durch mehrere Faktoren begrenzt. Erstens ist die Stichprobengröße begrenzt, wodurch gruppenspezifische Effekte nur eingeschränkt trennscharf beobachtbar sind. Zweitens setzt das strikte Zeitlimit den Fokus auf frühe Integrationsschritte und kann unvollständige Stabilisierung begünstigen. Drittens wurde GitHub Copilot als Assistenzumgebung in Kombination mit dem Modell Claude Sonnet 4.5 betrachtet, wodurch sich keine Aussagen zur Generalisierbarkeit über andere KI-Assistenzsysteme oder Modelle ableiten lassen. Viertens liegt keine Langzeitbeobachtung vor, sodass Lern- und Adaptationseffekte über Wochen oder Monate nicht erfasst werden.

Aus diesen Limitationen ergeben sich unmittelbare Ansatzpunkte für weitere Forschung. Sinnvoll sind längere Beobachtungsfenster mit wiederholten Sessions, um Stabilisierung, Refactoring und Validierungsroutinen erfassen zu können. Ebenso sind Designs relevant, die Prozessdaten stärker erfassen, etwa Testlauf-Sequenzen, Review-Handlungen oder Kontextbereitstellung im Editor. Schließlich ist es für künftige Arbeiten relevant zu prüfen, inwieweit die beobachteten Muster über unterschiedliche KI-Assistenzsysteme und Modellkonfigurationen hinweg stabil sind, ohne daraus unmittelbare Leistungsrangfolgen abzuleiten.

Kapitel 8 fasst die zentralen Erkenntnisse zusammen und leitet daraus die abschließenden Antworten auf die Forschungsfragen ab.

8 Fazit und Schlussfolgerungen

8.1 Ziel und Einordnung des Kapitels

Kapitel 8 schließt die Arbeit ab, indem die in Kapitel 6 berichteten Ergebnisse zusammengeführt und die in Kapitel 7 diskutierten Einordnungen in prägnante Schlussfolgerungen überführt werden. Im Unterschied zu Kapitel 6 (deskriptiv) und Kapitel 7 (theoriegeleitet) dient dieses Kapitel der abschließenden Synthese und der expliziten Beantwortung der Forschungsfragen. Es werden keine neuen Daten eingeführt und keine zusätzlichen Metriken berechnet.

Übergeordnetes Ziel der Arbeit war die Evaluation eines interaktiven KI-Assistenzsystems im Vibe-Coding-Kontext in einer kontrollierten Brownfield-Sandbox. Im Studiensetup wurde – konsistent zur Begriffsklärung in Kapitel 1 – zwischen Assistenzumgebung (Tool), Modell (LLM) und KI-Assistenzsystem als funktionaler Einheit unterschieden.

8.2 Beantwortung der Forschungsfragen

8.2.1 Forschungsfrage 1: Wie hängt die Entwicklerkompetenz mit dem Erreichen der Pipeline-Status Attempted, Implemented_{static} und Verified zusammen?

Kurzantwort. Attempted und Implemented_{static} sind über die Kompetenzgruppen hinweg als Artefaktspuren sichtbar; Verified als testgebundene Stufe wird am seltensten erreicht.

Verdichtete Begründung. Kapitel 6 trennt die Evidenzebenen Attempted, Implemented_{static} und Verified. Attempted steht für Intention (Chat-Ebene), Implemented_{static} für White-Box-Artefakte (Codebasis), Verified für testgebundenes Black-Box-Verhalten und ist damit methodengebunden. Für die Trichterdarstellung wird in Kapitel 6 zusätzlich Implemented* als abgeleitete Evidenzstufe verwendet. Die Befunde aus Kapitel 6 und die Einordnung in Kapitel 7 sind damit vereinbar, dass Implementationsfortschritt nicht notwendigerweise in testgebundene Bestätigung übergeht. Kompetenz dient dabei als Analyseperspektive auf unterschiedliche Integrations- und Validierungsmodi, nicht als Leistungsbewertung.

8.2.2 Forschungsfrage 2: Welche Rolle spielen Code- und Test-Artefakte für die Bewertung der Arbeitsergebnisse eines KI-Assistenzsystems?

Kurzantwort. Code- und Test-Artefakte sind zentral, weil sie die Trennung zwischen sichtbarer Implementierung (Implemented_{static}) und testgebundener Bestätigung (Verified) operationalisieren und Chat-Signale methodisch ergänzen.

Verdichtete Begründung. Die Artefaktanalyse macht Implementationsfortschritt über Quellcodeänderungen, Surrogat-Signale (z.B. Linting) und strukturelle Spuren in der Codebasis nachvollziehbar. Testgebundene Ergebnisse bilden beobachtbares Verhalten unter den gegebenen Tests ab und sind damit zentral für die Bestätigung im Systemkontext. Die Pipeline-Logik liefert so eine nachvollziehbare Trennung von Intention, Implementationsartefakt und testgebundener Bestätigung (Kapitel 6) und wird in Kapitel 7 methodisch eingeordnet. Kompetenz ergänzt diese Sicht als Analyseperspektive, weil sich Artefaktspuren nicht sinnvoll in einem einzelnen, gruppenunabhängigen Signal verdichten lassen.

8.2.3 Forschungsfrage 3: Welche Chat-Interaktionsmuster treten im Vibe-Coding-Kontext auf, und wie sind sie für die Evaluation einzuordnen?

Kurzantwort. Im Vibe-Coding-Kontext variieren Chat-Interaktionsmuster deutlich; ihre Intensität ist weder als Effizienzmaß noch als Qualitätsmaß geeignet. Für die Evaluation sind Chat-Muster nur im Zusammenspiel mit Pipeline-Status sowie Code- und Test-Artefakten aussagekräftig.

Verdichtete Begründung. Die Chat-Artefakte zeigen unterschiedliche Prompt-Strukturen, wechselnde Interaktionsmodi und wiederkehrende Debugging-Schleifen. Diese Muster sind konsistent damit, dass Interaktion sowohl Exploration als auch Beschleunigung unterstützen kann. Eine hohe Interaktionsintensität kann dabei gleichermaßen aus Validierung, aus Fehlanpassungen an den Systemkontext oder aus iterativer Integration resultieren. Entsprechend ist die Einordnung von Chat-Mustern an die Pipeline-Logik zu koppeln. Attempted macht die Intention sichtbar, Implemented_{static} die resultierenden Artefakte und Verified die testgebundene Funktion im System. Die Kompetenzgruppen liefern hierfür den Kontext als Analyseperspektiven auf Nutzungsformen, ohne dass daraus eine Bewertung einzelner Personen abgeleitet wird.

8.3 Zentrale Erkenntnisse der Arbeit

Die Arbeit verdichtet sich in folgenden Befundlinien.

- **Kompetenz als Perspektive auf Nutzungstypen:** Unterschiede zeigen sich in Nutzungsformen des KI-Assistenzsystems, nicht als Wertung von Personen oder Gruppen.

- **Trennung von Intention, Artefakt und Bestätigung:** Attempted, Implemented_{static} und Verified bilden unterschiedliche Ebenen desselben Arbeitsprozesses ab und fallen nicht notwendigerweise zusammen.
- **Validierung und Integration als Engpass:** Sichtbare Implementationsartefakte sind häufig erreichbar, während testgebundene Bestätigung die methodisch engste Stufe darstellt.
- **Chat-Interaktion ist kein Ersatz für Ergebnisprüfung:** Interaktionsintensität und Prompt-Formen sind ohne Artefakt- und Testbezug nicht belastbar als Effizienz- oder Qualitätssignale.
- **Grenzen vereinfachter Produktivitätsnarrative:** Produktivität lässt sich in der untersuchten Umgebung nicht auf Generierungsgeschwindigkeit reduzieren, sondern ist entlang von Integrationsaufwand und Validierungsarbeit einzuordnen.
- **Relevanz von Triangulation statt Einzelmetriken:** Belastbare Einordnung entsteht durch das Zusammenspiel von Pipeline-Status, Code- und Test-Artefakten sowie Chat-Mustern.

8.4 Beitrag der Arbeit

Wissenschaftlicher Beitrag. Die Arbeit liefert eine strukturierte Einordnung der Evaluation eines KI-Assistenzsystems im Vibe-Coding-Kontext, indem Kompetenzgruppen als Analyseperspektiven mit Artefaktspuren und Prozesssignalen zusammengeführt werden.

Methodischer Beitrag. Die Arbeit operationalisiert Aufgabenerfolg über die Pipeline-Logik (Attempted, Implemented_{static}, Verified; sowie Implemented* als abgeleitete Evidenzstufe) und verknüpft diese systematisch mit der Triangulation mehrerer Artefaktquellen. Damit liegt ein Evaluationsschema vor, das die Trennung zwischen intendierter Bearbeitung, sichtbarer Implementierung und testgebundener Bestätigung explizit und nachvollziehbar macht.

Konzeptioneller Beitrag. Die Arbeit betont Kompetenz als Perspektive auf Nutzungstypen und verschiebt den Fokus von einer rein toolzentrierten Betrachtung hin zu der Frage, wie KI-Assistenzsysteme in konkreten Entwicklungspraktiken genutzt und geprüft werden.

Empirischer Beitrag. Die Sandbox-Studie in einer Brownfield-Umgebung liefert eine Artefaktbasis aus Code-, Test- und Chat-Spuren, anhand derer die Pipeline-Status operationalisiert und gruppenbezogen eingeordnet werden können (Kapitel 6/7).

Praktischer Beitrag. Für Organisationen ergibt sich ein Rahmen zur Einführung und Evaluation von KI-Coding-Tools, der Qualifizierung, Review-Praxis und testbasierte Absicherung als Bestandteile einer kontrollierten Nutzung sichtbar macht.

8.5 Abschließendes Fazit

Die Arbeit zeigt, dass die Evaluation KI-gestützter Softwareentwicklung im Vibe-Coding-Kontext eine klare Trennung zwischen Intention, Implementationsartefakt und testgebundener Bestätigung erfordert. Die Kombination aus Pipeline-Logik, Artefaktanalyse und Chat-Auswertung erlaubt eine konsistente, methodisch abgesicherte Zusammenführung der Befunde (Kapitel 6) und ihrer Einordnung (Kapitel 7).

Im Ergebnis ist der Nutzen im untersuchten Brownfield-Setting nicht als automatische Eigenschaft eines Tools zu lesen, sondern als Zusammenspiel von Integrationsaufwand, Validierungsarbeit und Nutzungstypen im Kompetenzkontext.

Absicherung (Group comparability & Confounder). Die Schlussfolgerungen sind an die in Kapitel 6 berichteten Proxys (Pipeline-Status, Code-/Test-Surrogate, Chat-Muster) gebunden; mögliche Konfundierungen und die konservative Lesart werden in Abschnitt 7.1 (Kapitel 7) begründet. Entsprechend begründen die Befunde keine kausalen Effekte und keine generalisierbaren Rangfolgen zwischen Kompetenzgruppen. Die Aussagen beziehen sich auf Branches als Analyseeinheiten und auf aggregierte Muster, nicht auf eine Leistungsbewertung einzelner Personen.

Take-Home-Message. Für KI-gestützte Softwareentwicklung sind singuläre Produktivitäts- oder Proxy-Metriken nur begrenzt belastbar, weil Integrations- und Validierungsarbeit zentral bleibt (Kapitel 6/7). Robust wird die Evaluation durch die konsequente Trennung und Triangulation von Intention (Attempted), Implementationsartefakt (Implemented_{static}) und testgebundener Bestätigung (Verified). Kompetenz wird dabei durchgehend als Analyseperspektive genutzt, um unterschiedliche Nutzungs- und Validierungsmodi einzuordnen – ohne Leistungsrangfolgen abzuleiten. Die Kernaussage ist: Belastbare Schlussfolgerungen entstehen erst aus der gemeinsamen Betrachtung dieser drei Ebenen.

Literatur

- Anderson, John R. (1996). *The Architecture of Cognition*. Mahwah, NJ: Psychology Press.
- Barke, Shraddha, Michael B. James und Nadia Polikarpova (2023). „Grounded Copilot: How Programmers Interact with Code-Generating Models“. In: *Proceedings of the ACM on Programming Languages* 7.OOPSLA1, S. 1–27. DOI: 10.1145/3586030.
- Bavarian, Mohammad u. a. (2022). „Efficient Training of Language Models to Fill in the Middle“. In: *arXiv preprint arXiv:2207.14255*. DOI: 10.48550/arXiv.2207.14255.
- Brandtner, Matthias, Philipp Leitner und Cor-Paul Bezemer (2024). „Challenges of Brownfield Software Modernization in Practice: A Systematic Review“. In: *Journal of Systems and Software*. DOI: 10.1016/j.jss.2024.111743.
- Collins, Allan, John Seely Brown und Susan E. Newman (1989). „Cognitive Apprenticeship: Teaching the Crafts of Reading, Writing, and Mathematics“. In: *Knowing, Learning, and Instruction: Essays in Honor of Robert Glaser*, S. 453–494.
- Cook, Thomas D. und Donald T. Campbell (1979). *Quasi-Experimentation: Design & Analysis Issues for Field Settings*. Boston: Houghton Mifflin. ISBN: 978-0395307878.
- Dreyfus, Hubert L. und Stuart E. Dreyfus (1986). *Mind over Machine: The Power of Human Intuition and Expertise in the Era of the Computer*. New York: Free Press.
- Forsgren, Nicole, Margaret-Anne Storey und Thomas Zimmermann (2021). „SPACE: A Framework for Understanding Productivity in Software Engineering“. In: *Communications of the ACM* 64.6, S. 33–36. DOI: 10.1145/3454122.
- Green, Thomas R. G. (1989). *Cognitive Dimensions of Notations*. Cambridge University Press, S. 443–460.
- Horvitz, Eric (1999). „Principles of Mixed-Initiative User Interfaces“. In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, S. 159–166. DOI: 10.1145/302979.303029.
- ISO/IEC 25010:2011 *Systems and Software Engineering—System and Software Quality Models* (2011). Standard.
- Kahneman, Daniel (2011). *Thinking, Fast and Slow*. New York: Farrar, Straus und Giroux. ISBN: 978-0374275631.
- Lave, Jean und Etienne Wenger (1991). *Situated Learning: Legitimate Peripheral Participation*. Cambridge: Cambridge University Press.
- Lee, John D. und Katrina A. See (2004). „Trust in Automation: Designing for Appropriate Reliance“. In: *Human Factors* 46.1, S. 50–80. DOI: 10.1518/hfes.46.1.50.
- Martin, Robert C. (2009). *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ: Prentice Hall. ISBN: 978-0132350884.

- Mozannar, Hussein, Gagan Bansal, Adam Fourney und Eric Horvitz (2022). „Reading Between the Lines: Modeling User Behavior and Costs in AI-Assisted Programming“. In: *arXiv preprint arXiv:2210.14306*.
- Nadi, Sarah u. a. (2022). „Human-AI Collaboration in Software Engineering: Challenges and Opportunities“. In: *arXiv preprint arXiv:2206.01081*. DOI: 10.48550/arXiv.2206.01081.
- Parasuraman, Raja und Dietrich H. Manzey (2010). „Complacency and Bias in Human Interaction with Automation: An Attentional Integration“. In: *Human Factors* 52.3, S. 381–410. DOI: 10.1177/0018720810376055.
- Pearce, Henry u. a. (2022). „Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions“. In: *IEEE Symposium on Security and Privacy*. DOI: 10.1109/SP46214.2022.9833573.
- Peng, Sida, Eirini Kalliamvakou, Peter Cihon und Mert Demirel (2023). „The impact of AI on developer productivity: Evidence from GitHub Copilot“. In: *arXiv preprint arXiv:2302.06590*. DOI: 10.48550/arXiv.2302.06590.
- Saretsky, Gary (1972). „The OEO P.C. Experiment and the John Henry Effect“. In: *Phi Delta Kappan* 53.9, S. 579–581.
- Sweller, John (1988). „Cognitive Load During Problem Solving: Effects on Learning“. In: *Cognitive Science* 12.2, S. 257–285.
- Vaithilingam, Priyan u. a. (2022). „Expectations vs. Reality: Evaluating Code Completion Tools in Realistic Settings“. In: *CHI Conference on Human Factors in Computing Systems*. DOI: 10.1145/3491102.3502031.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser und Illia Polosukhin (2017). „Attention Is All You Need“. In: *Advances in Neural Information Processing Systems* 30, S. 5998–6008.
- Witte, Thomas u. a. (2023). „Context Windows and the Limits of AI-Assisted Programming“. In: *arXiv preprint arXiv:2308.10253*. DOI: 10.48550/arXiv.2308.10253.
- Wohlin, Claes, Per Runeson, Martin Höst, Magnus C Ohlsson, Björn Regnell und Anders Wesslén (2012). *Experimentation in Software Engineering*. Springer. ISBN: 978-3-642-29043-5. DOI: 10.1007/978-3-642-29044-2.